
SpectraLog

Adnan Ekici

Aug 25, 2026

GETTING STARTED

1	Quick Example	3
2	Features	5
3	Documentation	7
3.1	Installation	7
3.2	Quick Start	7
3.3	Basic Usage	8
3.4	Logger Configuration	9
3.5	JSON Logging	11
3.6	Rich Console Logging	12
3.7	Syslog Logging	13
3.8	Multiprocessing-Safe File Logging	16
3.9	Custom Log Levels	17
3.10	Disabling Logging in Tests	19
3.11	API Reference	19
3.12	Architecture	58
3.13	Contributing	64
4	Indices	71
	Index	73

SpectraLog is a configurable Python logging package providing colored console logging, rotating file logging, structured JSON logging, Rich console output, syslog integration, custom log levels, and multiprocessing-safe file logging. It provides a small public API while keeping formatter, handler, and runtime composition internally separated.

QUICK EXAMPLE

```
from spectralog import CreateSpectralLogger

logger = CreateSpectralLogger(
    debug_mode=True,
    log_file_name="application.log",
)

logger.info("Application started")
logger.warning("Configuration requires attention")
```


FEATURES

- Colored console logging
- Rotating file logging
- JSON Lines logging
- Rich console integration
- Syslog output
- Custom log levels
- Multiprocessing-safe file logging
- Process-local application logger
- Test-time logging suppression

DOCUMENTATION

3.1 Installation

3.1.1 Requirements

SpectraLog requires Python 3.11 or later.

3.1.2 Install from PyPI

Install the latest released version with pip:

```
python -m pip install spectralog
```

3.1.3 Verify the Installation

After installation, verify that SpectraLog can be imported:

```
import spectralog
```

3.1.4 Development Installation

Clone the repository and install the package in editable mode:

```
python -m pip install -e .
```

3.1.5 Documentation Dependencies

Install the documentation dependencies with:

```
python -m pip install -e ".[docs]"
```

The documentation can then be built with Sphinx.

3.2 Quick Start

3.2.1 Create the Application Logger

SpectraLog should normally be initialized once near the application's entry point.

```
from spectralog import CreateSpectraLogger

logger = CreateSpectraLogger()

logger.info("Application started")
```

By default, SpectraLog:

- logs INFO and higher messages;
- displays log messages in the console;
- includes timestamps;
- writes logs to the logs directory;
- creates a daily .log file.

3.2.2 Debug Logging

Enable DEBUG-level logging with:

```
logger = CreateSpectraLogger(
    debug_mode=True,
)

logger.debug("Debug information")
```

3.2.3 Access the Logger from Other Modules

Initialize SpectraLog once:

```
from spectralog import CreateSpectraLogger

CreateSpectraLogger()
```

Then retrieve the same application logger elsewhere:

```
from spectralog import get_logger

logger = get_logger()
logger.info("Processing request")
```

SpectraLog uses a process-local application logger singleton. Repeated calls to `get_logger` return the same logger instance.

3.3 Basic Usage

3.3.1 Standard Log Levels

SpectraLog provides the standard Python logging levels:

```
logger.debug("Debug message")
logger.info("Informational message")
logger.warning("Warning message")
```

(continues on next page)

(continued from previous page)

```
logger.error("Error message")
logger.critical("Critical message")
```

3.3.2 Exception Logging

Use exception inside an exception handler to include traceback information:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    logger.exception("Calculation failed")
```

3.3.3 Deferred Message Formatting

SpectraLog supports the standard logging interpolation mechanism:

```
user_name = "Ada"

logger.info(
    "User %s authenticated",
    user_name,
)
```

Using deferred interpolation avoids formatting a message before the logging system determines whether the record will be emitted.

3.3.4 Generic Log Method

A level can also be supplied explicitly:

```
logger.log(
    "WARNING",
    "Disk usage is high",
)
```

Numeric severity values are also supported:

```
logger.log(
    30,
    "Disk usage is high",
)
```

3.4 Logger Configuration

SpectraLog is configured through `spectralog.CreateSpectraLogger()`.

3.4.1 Core Configuration

```
logger = CreateSpectraLogger(
    debug_mode=False,
    show_datetime=True,
```

(continues on next page)

(continued from previous page)

```
show_line=False,  
show_folder_name=False,  
logs_directory="logs",  
log_file_name=None,  
save_logs=True,  
multiprocessing_safe=False,  
)
```

3.4.2 Logging Level

debug_mode=False uses INFO as the effective logging threshold.

debug_mode=True uses DEBUG.

3.4.3 File Location

The default logging directory is:

```
logs/
```

A custom directory may be supplied:

```
logger = CreateSpectraLogger(  
    logs_directory="/var/log/my-application",  
)
```

3.4.4 Custom File Name

```
logger = CreateSpectraLogger(  
    log_file_name="application.log",  
)
```

When no file name is supplied, SpectraLog generates a daily log file.

3.4.5 Source Information

Include source paths:

```
logger = CreateSpectraLogger(  
    show_folder_name=True,  
)
```

Include source line numbers:

```
logger = CreateSpectraLogger(  
    show_line=True,  
)
```

Both options may be enabled together:

```
logger = CreateSpectraLogger(  
    show_folder_name=True,
```

(continues on next page)

(continued from previous page)

```

    show_line=True,
)

```

3.4.6 Custom Formats

Use `console_format` or `file_format` when the generated format does not meet application requirements.

```

logger = CreateSpectraLogger(
    console_format="%(levelname)s | %(message)s",
    file_format="%(asctime)s | %(levelname)s | %(message)s",
)

```

3.4.7 Log Rotation

SpectraLog uses rotating file handlers.

```

logger = CreateSpectraLogger(
    max_bytes=10 * 1024 * 1024,
    backup_count=5,
)

```

`max_bytes` controls the maximum active file size before rollover.

`backup_count` controls the number of rotated files retained.

3.5 JSON Logging

SpectraLog can write structured JSON Lines log files.

3.5.1 Enable JSON Logging

```

from spectralog import CreateSpectraLogger
from spectralog import JsonLoggerConfiguration

logger = CreateSpectraLogger(
    log_file_name="application.log",
    json_logger_configuration=JsonLoggerConfiguration(),
)

```

When JSON logging is enabled, the resulting file uses the `.jsonl` extension.

3.5.2 Example Record

A record may resemble:

```

{
  "level": "INFO",
  "message": "Application started",
  "timestamp": "2026-08-25T12:30:00+00:00",
  "logger": "ApplicationLogger",
  "process_id": 12345,
  "process_name": "MainProcess",

```

(continues on next page)

(continued from previous page)

```
"thread_id": 123456789,  
"thread_name": "MainThread"  
}
```

3.5.3 Optional Metadata

Metadata fields can be disabled individually:

```
json_configuration = JsonLoggerConfiguration(  
    include_timestamp=True,  
    include_logger_name=False,  
    include_process_information=False,  
    include_thread_information=False,  
)
```

Level and message fields are always included.

3.6 Rich Console Logging

SpectraLog integrates with Rich for enhanced terminal output.

3.6.1 Enable Rich Output

```
from spectralog import CreateSpectraLogger  
from spectralog import RichConsoleConfiguration  
  
logger = CreateSpectraLogger(  
    rich_console_configuration=RichConsoleConfiguration(),  
)
```

3.6.2 Configuration

```
rich_configuration = RichConsoleConfiguration(  
    show_time=True,  
    show_level=True,  
    show_path=True,  
    rich_tracebacks=True,  
    markup=False,  
)
```

3.6.3 Rich Tracebacks

With `rich_tracebacks=True`, exceptions logged through `logger.exception` use Rich's enhanced traceback rendering.

3.6.4 Markup

Rich markup is disabled by default.

Enable it only when log messages intentionally contain Rich markup syntax:


```
RichConsoleConfiguration(
    markup=True,
)
```

3.7 Syslog Logging

SpectraLog can forward log records to a syslog server in addition to writing to the console and, when enabled, to local log files.

Syslog support is configured with *spectralog.SyslogConfiguration*.

3.7.1 Basic Usage

Create a syslog configuration and pass it to *spectralog.CreateSpectraLogger()*:

```
import socket

from spectralog import CreateSpectraLogger
from spectralog import SyslogConfiguration

logger = CreateSpectraLogger(
    syslog_configuration=SyslogConfiguration(
        host="localhost",
        port=514,
        socket_type=socket.SOCK_DGRAM,
    ),
)

logger.info("Application started")
```

3.7.2 Configuration

SyslogConfiguration supports the following options:

host

Hostname or IP address of the target syslog server.

Defaults to "localhost".

port

Network port used by the syslog server.

Defaults to 514.

facility

Syslog facility passed to Python's *logging.handlers.SysLogHandler*.

Defaults to *SysLogHandler.LOG_USER*.

socket_type

Socket type used for communication with the syslog server.

Common values are:

- *socket.SOCK_DGRAM* for UDP
- *socket.SOCK_STREAM* for TCP

Defaults to `socket.SOCK_DGRAM`.

3.7.3 UDP Example

UDP is the default transport:

```
import socket

from spectralog import CreateSpectraLogger
from spectralog import SyslogConfiguration

logger = CreateSpectraLogger(
    syslog_configuration=SyslogConfiguration(
        host="127.0.0.1",
        port=514,
        socket_type=socket.SOCK_DGRAM,
    ),
)

logger.warning("Disk usage is high")
```

3.7.4 TCP Example

TCP can be enabled by changing the socket type:

```
import socket

from spectralog import CreateSpectraLogger
from spectralog import SyslogConfiguration

logger = CreateSpectraLogger(
    syslog_configuration=SyslogConfiguration(
        host="logs.example.com",
        port=514,
        socket_type=socket.SOCK_STREAM,
    ),
)

logger.error("Service failed")
```

3.7.5 Combined Logging

Syslog can be used together with console and file logging.

For example:

```
import socket

from spectralog import CreateSpectraLogger
from spectralog import SyslogConfiguration

logger = CreateSpectraLogger(
    logs_directory="logs",
    log_file_name="application.log",
```

(continues on next page)

(continued from previous page)

```

    save_logs=True,
    syslog_configuration=SyslogConfiguration(
        host="localhost",
        port=514,
        socket_type=socket.SOCK_DGRAM,
    ),
)

logger.info("Application started")

```

In this configuration, the same log record can be emitted to:

- the console;
- the configured log file;
- the syslog server.

3.7.6 Formatting

Syslog records use SpectraLog's plain-text formatter rather than the colored console formatter.

This prevents terminal-specific color escape sequences from being transmitted to the syslog server.

If source-path information is enabled, SpectraLog also applies its relative path filter to syslog records.

For example:

```

logger = CreateSpectraLogger(
    show_folder_name=True,
    show_line=True,
    syslog_configuration=SyslogConfiguration(),
)

```

3.7.7 Custom Formats

A custom file format also affects syslog formatting because SpectraLog uses the plain-text file formatter for syslog output.

For example:

```

logger = CreateSpectraLogger(
    file_format="%(levelname)s | %(message)s",
    syslog_configuration=SyslogConfiguration(),
)

```

3.7.8 Custom Log Levels

Custom SpectraLog levels can also be sent through syslog.

```

logger.add_log_level(
    name="NOTICE",
    color="cyan",
    severity=35,
)

```

(continues on next page)

```
logger.notice("Deployment completed")
```

The configured syslog handler receives the record using the custom numeric severity registered with SpectraLog.

3.7.9 Network Considerations

SpectraLog delegates syslog transport to Python's `logging.handlers.SysLogHandler`.

The target syslog server must therefore be reachable from the application environment and configured to accept the selected transport.

For UDP, verify that the server accepts datagrams on the configured port.

For TCP, verify that the server is listening for stream connections on the configured port.

Firewalls, container networking, host networking rules, and remote syslog server configuration may affect delivery.

3.7.10 Example Configuration

A more complete configuration may look like:

```
import socket

from spectralog import CreateSpectraLogger
from spectralog import SyslogConfiguration

logger = CreateSpectraLogger(
    debug_mode=True,
    show_datetime=True,
    show_folder_name=True,
    show_line=True,
    logs_directory="logs",
    log_file_name="application.log",
    save_logs=True,
    syslog_configuration=SyslogConfiguration(
        host="192.168.1.50",
        port=514,
        socket_type=socket.SOCK_DGRAM,
    ),
)

logger.info("Service initialized")
```

3.7.11 See Also

See `spectralog.SyslogConfiguration` for the complete configuration API.

See `spectralog.CreateSpectraLogger()` for the main logger creation API.

3.8 Multiprocessing-Safe File Logging

SpectraLog supports queue-based file logging through `multiprocessing_safe=True`.

```
logger = CreateSpectraLogger(
    multiprocessing_safe=True,
)
```

3.8.1 Architecture

In this mode, records flow through:

```

ApplicationLogger
  |
  v
QueueHandler
  |
  v
Multiprocessing Queue
  |
  v
QueueListener
  |
  v
RotatingFileHandler

```

The application logger does not write directly to the file handler.

3.8.2 Runtime Lifecycle

The multiprocessing logging runtime is started automatically when the application logger is created.

It is stopped during `ApplicationLogger.shutdown` and also registered for normal interpreter shutdown.

3.8.3 Independent Processes

The multiprocessing runtime belongs to the logger instance that created it.

Independently initialized Python processes do not automatically share the same queue or queue listener. Applications that initialize SpectraLog separately in multiple processes should use separate log files unless they explicitly coordinate logging through shared infrastructure.

3.9 Custom Log Levels

SpectraLog allows custom logging levels to be registered at runtime.

3.9.1 Register a Level

```
logger.add_log_level(
    name="NOTICE",
    color="cyan",
    severity=35,
)
```

3.9.2 Dynamic Logging Methods

After registration, the level becomes available as a dynamic logger method:

```
logger.notice(  
    "Deployment completed",  
)
```

The level may also be used through log:

```
logger.log(  
    "NOTICE",  
    "Deployment completed",  
)
```

3.9.3 Naming Rules

Custom level names:

- must not be empty;
- must begin with a letter;
- may contain letters, numbers, and underscores;
- are normalized to uppercase.

For example:

```
logger.add_log_level(  
    name="audit_event",  
    color="cyan",  
    severity=35,  
)
```

is registered as:

```
AUDIT_EVENT
```

3.9.4 Severity Rules

Severity values must:

- be integers;
- not be boolean values;
- be greater than zero;
- not duplicate an existing registered severity.

3.9.5 Colors

Colors must be supported by colorlog.

3.10 Disabling Logging in Tests

Applications often exercise code that initializes SpectraLog during unit or integration tests. Creating console handlers, files, multiprocessing infrastructure, or shutdown callbacks may be undesirable in those tests.

SpectraLog provides `disable_application_logging` for this purpose.

3.10.1 Basic Usage

```
import unittest

from spectralog import CreateSpectraLogger
from spectralog import disable_application_logging
from spectralog import get_logger

@disable_application_logging
class UnitTestApplication(unittest.TestCase):
    def test_application_behavior(self) -> None:
        CreateSpectraLogger(
            log_file_name="application.log",
        )

        logger = get_logger()
        logger.info("This will not produce logging output")

        self.assertTrue(True)
```

3.10.2 Behavior

While the decorated test class executes:

- SpectraLog's normal logger build operation is replaced with a disabled logger;
- no SpectraLog file handler is created;
- no Rich handler is created;
- no syslog handler is created;
- no multiprocessing logging runtime is created;
- SpectraLog `atexit` registration is suppressed;
- each test method receives isolated application-logger singleton state.

The application can continue to call `CreateSpectraLogger`, `get_logger`, and normal logging methods without requiring test-specific changes to the application code.

3.11 API Reference

This section contains the generated reference documentation for SpectraLog's public interfaces and internal components.

3.11.1 Public API

Logger Creation

```
spectralog.CreateSpectraLogger(debug_mode=False, show_datetime=True, show_line=False,  
                                show_folder_name=False, logs_directory='logs', log_file_name=None,  
                                save_logs=True, multiprocessing_safe=False, syslog_configuration=None,  
                                rich_console_configuration=None, json_logger_configuration=None,  
                                console_format=None, file_format=None, date_format='%Y-%m-%d  
                                %H:%M:%S', max_bytes=20 * 1024**2, backup_count=1)
```

Create and configure the process-wide SpectraLog application logger.

This function is the primary entry point for initializing SpectraLog. It creates a `LoggerConfiguration`, composes the required logger infrastructure, and initializes the process-local `ApplicationLogger` singleton.

The application logger should normally be initialized once near the application's entry point. After initialization, other modules should use `get_logger()` to retrieve the existing logger rather than calling `CreateSpectraLogger` again.

Console logging is always configured. When `save_logs` is enabled, SpectraLog additionally creates a file handler using either plain-text or JSON Lines formatting. Rich console output, syslog output, and multiprocessing-safe file logging can be enabled independently through their corresponding configuration options.

When `json_logger_configuration` is supplied, file logging uses JSON Lines format and the resolved log file has a `.jsonl` extension. Otherwise, standard text formatting is used.

When `rich_console_configuration` is supplied, the Rich console handler is used instead of the standard colored console handler.

Parameters

- **debug_mode** (*bool*) – Enables debug-level logging when True. The logger and its configured handlers use `logging.DEBUG` as their minimum level. When False, the minimum level is `logging.INFO`. Defaults to False.
- **show_datetime** (*bool*) – Includes the formatted timestamp in automatically generated plain-text console and file formats when True. This option does not affect an explicitly supplied `console_format` or `file_format`. Defaults to True.
- **show_line** (*bool*) – Includes the source line number in automatically generated plain-text formats when True. When `show_folder_name` is also enabled, the line number is appended to the relative source path. Otherwise, it is rendered as a standalone line component. Defaults to False.
- **show_folder_name** (*bool*) – Includes the source file path relative to the detected project root in automatically generated plain-text formats when True. The relative path is populated by SpectraLog's relative-path logging filter. Defaults to False.
- **logs_directory** (*str* / *Path*) – Directory in which log files are created. A string or `pathlib.Path` may be supplied. The directory is expanded, resolved, and created when file logging is enabled. Defaults to "logs".
- **log_file_name** (*str* / *None*) – Optional custom log file name. When omitted, SpectraLog generates a daily file name from the current date. When JSON logging is enabled, the file suffix is changed to `.jsonl` regardless of the supplied suffix. Defaults to None.
- **save_logs** (*bool*) – Enables persistent file logging when True. When False, SpectraLog does not resolve or create a log file and no file handler is attached. Console and optionally configured syslog logging remain available. Defaults to True.

- **multiprocessing_safe** (*bool*) – Routes file log records through SpectraLog’s multiprocessing queue and listener infrastructure when *True*. This prevents the application’s logger from writing directly to the file handler in the initializing process and ensures queued records are flushed during logger shutdown. Defaults to *False*.

This option applies to the multiprocessing runtime created by this logger instance. Independently initialized processes do not automatically share the same queue or listener.

- **syslog_configuration** (*SyslogConfiguration* / *None*) – Optional *SyslogConfiguration* describing the remote syslog host, port, facility, and socket type. When supplied, a syslog handler is attached in addition to the configured console and file handlers. Defaults to *None*.
- **rich_console_configuration** (*RichConsoleConfiguration* / *None*) – Optional *RichConsoleConfiguration* controlling Rich console rendering. When supplied, SpectraLog uses a Rich console handler instead of its standard colored console handler. Defaults to *None*.
- **json_logger_configuration** (*JsonLoggerConfiguration* / *None*) – Optional *JsonLoggerConfiguration* controlling structured file logging. When supplied, file records are serialized as JSON Lines and the log file uses the *.jsonl* extension. Defaults to *None*.
- **console_format** (*str* / *None*) – Optional explicit format string for the standard colored console formatter. When supplied, it overrides the console format that would otherwise be generated from *show_datetime*, *show_line*, and *show_folder_name*. Defaults to *None*.
This setting applies to the standard console formatter and is not used to format Rich console output.
- **file_format** (*str* / *None*) – Optional explicit format string for plain-text file logging. When supplied, it overrides the automatically generated file format. JSON logging uses its JSON formatter instead of this format string. Defaults to *None*.
- **date_format** (*str*) – Date and time format passed to logging formatters that render an *%(asctime)s* value. Defaults to *"%Y-%m-%d %H:%M:%S"*.
- **max_bytes** (*int*) – Maximum size, in bytes, of a plain or JSON log file before the rotating file handler performs a rollover. Defaults to *20 * (1024**2)*, or 20 MiB.
- **backup_count** (*int*) – Number of rotated backup log files retained by the rotating file handler. Defaults to 1.

Returns

The initialized process-local SpectraLog application logger. Subsequent calls to *get_logger()* return this same *ApplicationLogger* instance.

Return type

ApplicationLogger

Raises

SpectraApplicationLoggerReconfigurationError – If *CreateSpectraLogger* is called after the application logger has already been initialized. SpectraLog does not permit the existing process-local singleton to be reconfigured.

Example

Initialize SpectraLog once at the application entry point:

```
from spectralog import CreateSpectraLogger

logger = CreateSpectraLogger(
    debug_mode=True,
    logs_directory="logs",
    log_file_name="application.log",
)

logger.info("Application started")
```

Retrieve the same logger elsewhere in the application with `get_logger()` rather than initializing it again.

Logger Access

`spectralog.get_logger()`

Return the process-local SpectraLog application logger.

Returns the existing `ApplicationLogger` singleton when SpectraLog has already been initialized. This is the recommended way for application modules to access the logger after the initial call to `CreateSpectraLogger()`.

If the application logger has not yet been initialized, a default logger is created using SpectraLog's default configuration.

Returns

The process-local SpectraLog application logger instance. Repeated calls return the same singleton instance for the lifetime of the process.

Return type

ApplicationLogger

Example

Initialize SpectraLog once at application startup:

```
from spectralog import CreateSpectraLogger

CreateSpectraLogger(
    debug_mode=True,
    log_file_name="application.log",
)
```

Retrieve the existing logger from another module:

```
from spectralog import get_logger

logger = get_logger()
logger.info("Processing started")
```

Testing

`spectralog.disable_application_logging(test_class)`

Disable SpectraLog output and logging side effects for a unittest class.

This decorator is intended for test suites that exercise application code which initializes or retrieves SpectraLog through its normal public API, but where actual logging infrastructure is undesirable.

While the decorated test class runs, SpectraLog's logger builder is replaced with a build result containing a disabled standard-library logger. The disabled logger has no active handlers, does not propagate records, does not create log files, and does not initialize a multiprocessing logging runtime.

The decorator also suppresses SpectraLog's `atexit` registration so that test execution does not accumulate shut-down callbacks.

Logging infrastructure is disabled for the lifetime of the decorated test class, while the `ApplicationLogger` singleton is isolated separately for each individual test method. Before each test, the current singleton instance is preserved and cleared. After each test, any logger created during that test is shut down and the previously existing singleton is restored.

Existing `setUpClass`, `tearDownClass`, `setUp`, and `tearDown` implementations on the decorated test class are preserved and invoked as part of the wrapped lifecycle. Logging patches are also cleaned up if class setup fails, and singleton state is restored if per-test setup fails.

Application code may continue to call `CreateSpectraLogger()`, `get_logger()`, and normal logging methods while this decorator is active. Those calls operate against a valid `ApplicationLogger`, but no SpectraLog console, file, Rich, syslog, or multiprocessing output is produced.

Parameters

test_class (`TestCaseClass`) – The `unittest.TestCase` subclass for which SpectraLog output and logging side effects should be disabled.

Returns

The same test class with wrapped class-level and per-test lifecycle methods that manage disabled logging and singleton isolation.

Return type

`TestCaseClass`

Example

Apply the decorator directly to a unittest class:

```
import unittest

from spectralog import CreateSpectraLogger
from spectralog import disable_application_logging
from spectralog import get_logger

@disable_application_logging
class UnitTestExperimentService(unittest.TestCase):
    def test_experiment_execution(self) -> None:
        CreateSpectraLogger(
            log_file_name="application.log",
            save_logs=True,
        )

        logger = get_logger()
        logger.info("Experiment started")

        self.assertTrue(True)
```

The application may initialize and use SpectraLog normally inside the test, but no log file or other logging infrastructure is created.

Note

The decorator is designed specifically for `unittest.TestCase` subclasses.

Singleton isolation occurs per test method rather than only once per test class. This allows multiple tests in the same decorated class to call `CreateSpectraLogger()` independently without triggering SpectraLog's reconfiguration protection.

3.11.2 Configuration API

JSON Logging

```
class spectralog.JsonLoggerConfiguration(include_timestamp=True, include_logger_name=True,  
                                         include_process_information=True,  
                                         include_thread_information=True)
```

Configure the fields included in SpectraLog JSON log records.

This immutable configuration object controls which optional metadata fields are emitted by SpectraLog's JSON formatter.

The log level and message are always included in each JSON log record. The options defined here only control additional metadata such as timestamps, logger names, process information, and thread information.

Instances are frozen and use slots, preventing configuration values from being modified after creation or arbitrary attributes from being added dynamically.

Parameters

- `include_timestamp` (*bool*)
- `include_logger_name` (*bool*)
- `include_process_information` (*bool*)
- `include_thread_information` (*bool*)

`include_timestamp`

Includes a timestamp field in each JSON log record when True. Defaults to True.

Type

bool

`include_logger_name`

Includes the originating logger name in each JSON log record when True. Defaults to True.

Type

bool

`include_process_information`

Includes process-related metadata, such as the process identifier and process name, in each JSON log record when True. Defaults to True.

Type

bool

`include_thread_information`

Includes thread-related metadata, such as the thread identifier and thread name, in each JSON log record when True. Defaults to True.

Type

bool

Example

Configure JSON logging with selected metadata disabled:

```

from spectralog import JsonLoggerConfiguration

json_configuration = JsonLoggerConfiguration(
    include_timestamp=True,
    include_logger_name=False,
    include_process_information=False,
    include_thread_information=False,
)

```

Rich Console

```

class spectralog.RichConsoleConfiguration(show_time=True, show_level=True, show_path=True,
                                         rich_tracebacks=True, markup=False)

```

Configure SpectraLog's Rich-based console output.

This immutable configuration object controls the presentation and traceback behavior of the Rich console handler used by SpectraLog.

Providing an instance of this configuration when creating a SpectraLog logger enables Rich console rendering instead of the standard colored console handler. These options affect console output only and do not change file, JSON, or syslog formatting.

Instances are frozen and use slots, preventing configuration values from being modified after construction or arbitrary attributes from being added dynamically.

Parameters

- **show_time** (*bool*)
- **show_level** (*bool*)
- **show_path** (*bool*)
- **rich_tracebacks** (*bool*)
- **markup** (*bool*)

show_time

Displays Rich's timestamp column for each console log record when True. Defaults to True.

Type

bool

show_level

Displays the log level column in Rich console output when True. Defaults to True.

Type

bool

show_path

Displays source path information in Rich console output when True. Defaults to True.

Type

bool

rich_tracebacks

Enables Rich-formatted exception tracebacks when True. When enabled, exceptions logged with traceback information are rendered using Rich's enhanced traceback presentation. Defaults to True.

Type

bool

markup

Enables Rich markup processing within log messages when True. When False, Rich markup syntax contained in application messages is treated as ordinary text. Defaults to False.

Type

bool

Example

Configure compact Rich console output without timestamps or source paths:

```
from spectralog import RichConsoleConfiguration

rich_configuration = RichConsoleConfiguration(
    show_time=False,
    show_level=True,
    show_path=False,
    rich_tracebacks=True,
    markup=False,
)
```

Syslog

```
class spectralog.SyslogConfiguration(host='localhost', port=514, facility=1,
                                     socket_type=SocketKind.SOCK_DGRAM)
```

Configure SpectraLog's syslog handler.

This immutable configuration object defines the network destination and transport settings used when SpectraLog forwards log records to a syslog server.

Providing an instance of this configuration when creating a SpectraLog logger enables syslog output in addition to any configured console and file handlers.

Instances are frozen and use slots, preventing configuration values from being modified after construction or arbitrary attributes from being added dynamically.

Parameters

- **host** (*str*)
- **port** (*int*)
- **facility** (*int*)
- **socket_type** (*SocketKind*)

host

Hostname or IP address of the target syslog server. Defaults to "localhost".

Type

str

port

Network port used by the syslog server. The conventional syslog port is 514. Defaults to 514.

Type

int

facility

Syslog facility value passed to `logging.handlers.SysLogHandler`. Defaults to `logging.handlers.SysLogHandler.LOG_USER`.

Type

int

socket_type

Socket type used to communicate with the syslog server. Common values are `socket.SOCK_DGRAM` for UDP and `socket.SOCK_STREAM` for TCP. Defaults to `socket.SOCK_DGRAM`.

Type

socket.SocketKind

Example

Configure UDP syslog forwarding to a remote server:

```
import socket

from spectralog import SyslogConfiguration

syslog_configuration = SyslogConfiguration(
    host="192.168.1.50",
    port=514,
    socket_type=socket.SOCK_DGRAM,
)
```

Configure TCP syslog forwarding:

```
import socket

from spectralog import SyslogConfiguration

syslog_configuration = SyslogConfiguration(
    host="logs.example.com",
    port=514,
    socket_type=socket.SOCK_STREAM,
)
```

3.11.3 Exceptions

SpectraLog provides package-specific exceptions for logger lifecycle errors, custom log-level validation, and registry operations.

All SpectraLog-specific exceptions inherit from `spectralog.exceptions.exceptions.SpectraLogError`.

Base Exception

exception `spectralog.exceptions.exceptions.SpectraLogError`

Base exception for errors raised by SpectraLog's application logger.

All SpectraLog-specific logger lifecycle, configuration, and log-level validation exceptions inherit from this class.

Catching this exception allows callers to handle any SpectraLog application logger error through a single common base type.

Application Logger Exceptions

exception

`spectralog.exceptions.exceptions.SpectraApplicationLoggerAlreadyInitializedError`

Raised when `ApplicationLogger` is instantiated outside its managed lifecycle.

`ApplicationLogger` is designed to be created only through SpectraLog's singleton initialization path. Direct construction without the internal construction token raises this exception.

Applications should initialize logging through `CreateSpectraLogger()` or retrieve the existing logger through `get_logger()`.

exception `spectralog.exceptions.exceptions.SpectraApplicationLoggerReconfigurationError`

Raised when an initialized application logger is configured again.

SpectraLog uses a process-local `ApplicationLogger` singleton. Once the singleton has been initialized, supplying a new logger builder or log-level registry is treated as an attempt to reconfigure the active logger and raises this exception.

Applications should configure SpectraLog once during startup and use `get_logger()` for subsequent access.

Log-Level Registry Exceptions

exception `spectralog.exceptions.exceptions.SpectraLogLevelAlreadyExistsError`

Raised when attempting to register a log level that already exists.

A log-level name must uniquely identify a single registered level within SpectraLog's log-level registry. Registering another level with an existing name raises this exception.

exception `spectralog.exceptions.exceptions.SpectraLogLevelNotFoundError`

Raised when a requested log level is not registered.

This exception indicates that SpectraLog could not resolve the requested log-level name from its active log-level registry.

The level must be one of SpectraLog's predefined levels or a custom level previously registered with `ApplicationLogger.add_log_level()`.

Validation Exceptions

exception `spectralog.exceptions.exceptions.InvalidSpectraLogLevelNameError`

Raised when a log-level name fails SpectraLog's validation requirements.

This exception indicates that the supplied name cannot be used as a valid SpectraLog log-level identifier.

The exact accepted naming rules are defined by the log-level registry's validation logic.

exception `spectralog.exceptions.exceptions.InvalidSpectraLogLevelSeverityError`

Raised when a log-level severity fails SpectraLog's validation requirements.

Custom log levels must use a valid integer severity accepted by the log-level registry. This exception is raised when the supplied severity violates those requirements.

The exact accepted severity constraints are defined by the log-level registry's validation logic.

exception `spectralog.exceptions.exceptions.InvalidSpectraLogColorError`

Raised when a log-level color fails SpectraLog's validation requirements.

Custom log levels may define a color used by compatible colored console formatters. This exception is raised when the supplied color is not accepted by SpectraLog's log-level registry.

The exact supported color values are defined by the registry's color validation logic.

Handling SpectraLog Errors

Applications may catch a specific exception when different failure conditions require different behavior.

For example:

```
from spectralog import CreateSpectraLogger
from spectralog.exceptions.exceptions import (
    SpectraApplicationLoggerReconfigurationError,
)

try:
    CreateSpectraLogger()
    CreateSpectraLogger()
except SpectraApplicationLoggerReconfigurationError:
    print("SpectraLog has already been configured.")
```

Catching All SpectraLog Errors

All package-specific exceptions inherit from `SpectraLogError`.

This allows applications to handle any SpectraLog-specific failure through one base exception when individual error types do not need separate handling.

```
from spectralog.exceptions.exceptions import SpectraLogError

try:
    ...
except SpectraLogError as exception:
    print(f"SpectraLog error: {exception}")
```

Application Logger Lifecycle**`SpectraApplicationLoggerAlreadyInitializedError`**

Raised when `ApplicationLogger` is instantiated outside SpectraLog's managed singleton lifecycle.

`SpectraApplicationLoggerReconfigurationError`

Raised when configuration dependencies are supplied after the process-local application logger has already been initialized.

Custom Log Levels

SpectraLogLevelAlreadyExistsError

Raised when a custom log-level name is already registered or its numeric severity is already assigned to another level.

SpectraLogLevelNotFoundError

Raised when SpectraLog cannot find a requested log-level name in the active registry.

InvalidSpectraLogLevelNameError

Raised when a custom log-level name does not satisfy SpectraLog's naming requirements.

InvalidSpectraLogLevelSeverityError

Raised when a custom severity is not a positive integer or is otherwise invalid.

InvalidSpectraLogColorError

Raised when a custom log-level color is not supported by the configured color backend.

See Also

See [Custom Log Levels](#) for custom log-level registration and validation behavior.

See [Public API](#) for the main SpectraLog public API.

3.11.4 Internal API

This section documents SpectraLog's internal implementation components.

These classes support logger construction, formatting, handler creation, log-level management, file-path resolution, and multiprocessing runtime management.

They are primarily intended for contributors and advanced users. They should not be considered part of the stable public API unless explicitly documented elsewhere.

Application Logger

class `spectralog.core.logger.ApplicationLogger`(*logger_builder*, *log_level_registry*,
construction_token)

Bases: `object`

Provide the process-local SpectraLog application logger.

`ApplicationLogger` is the central logging facade exposed by SpectraLog. It wraps a configured `logging.Logger`, provides the standard logging methods, supports dynamically registered custom log levels, manages the optional multiprocessing logging runtime, and enforces process-local singleton semantics.

Instances must not be constructed directly. Applications should initialize SpectraLog through `CreateSpectraLogger()` or retrieve the current logger through `get_logger()`.

The singleton is protected by an internal lock so that initialization remains safe when multiple threads attempt to obtain the application logger concurrently.

When the configured logger uses multiprocessing-safe file logging, the associated runtime is started automatically during initialization and stopped when `shutdown()` is called. `shutdown` is also registered with `atexit` so that queued log records can be flushed during normal interpreter termination.

Custom log levels registered through `add_log_level()` become available as dynamic methods. For example, registering a NOTICE level allows `logger.notice("message")` to be used in the same manner as the built-in logging methods.

Parameters

- **logger_builder** (`LoggerBuilder`)
- **log_level_registry** (`LogLevelRegistry`)
- **construction_token** (*object*)

__init__ (*logger_builder, log_level_registry, construction_token*)

Initialize the application logger from a prepared logger builder.

Construction is restricted to the internal singleton initialization path. A private construction token is required to prevent callers from bypassing `get_instance()` and creating independent `ApplicationLogger` objects.

The supplied builder creates the underlying `logging.Logger` and returns its associated file and multiprocessing runtime metadata. If a multiprocessing logging runtime is present, it is started immediately.

The logger's `shutdown()` method is registered with `atexit`, and a warning is emitted when the builder reports that a new or empty log file has been created.

Parameters

- **logger_builder** (`LoggerBuilder`) – Builder responsible for constructing the underlying configured `logging.Logger` and its associated runtime metadata.
- **log_level_registry** (`LogLevelRegistry`) – Registry containing the standard and custom log levels known to this application logger.
- **construction_token** (*object*) – Internal token used to verify that construction was initiated through SpectraLog's singleton lifecycle.

Raises

`SpectraApplicationLoggerAlreadyInitializedError` – If construction is attempted directly without SpectraLog's internal construction token.

Return type

None

classmethod **get_instance** (*logger_builder=None, log_level_registry=None*)

Return or initialize the process-local application logger singleton.

If the application logger has not yet been created, this method initializes it using the supplied builder and log-level registry. Missing dependencies are created automatically using SpectraLog's default configuration.

If an instance already exists, calling this method without configuration arguments simply returns that existing instance.

Supplying a logger builder or log-level registry after initialization is treated as an attempt to reconfigure the singleton and is rejected.

Singleton initialization is protected by an internal lock, preventing multiple threads from independently creating application logger instances.

Parameters

- **logger_builder** (`LoggerBuilder` / `None`) – Optional builder used to construct the underlying logger during first initialization. When omitted, a builder using SpectraLog's default configuration is created automatically. Defaults to `None`.
- **log_level_registry** (`LogLevelRegistry` / `None`) – Optional registry containing the log levels available to the application logger. When omitted during first initialization, a new `LogLevelRegistry` is created automatically. Defaults to `None`.

Raises

SpectraApplicationLoggerReconfigurationError – If configuration dependencies are supplied after the singleton has already been initialized.

Returns

The process-local application logger singleton.

Return type

ApplicationLogger

add_log_level(*name, color, severity*)

Register a custom log level and refresh compatible console formatters.

Adds the supplied level definition to the application's `LogLevelRegistry`. Once registered, the level can be used through `log()` by name or through a dynamically resolved method whose name matches the registered level.

After registration, compatible console formatter color mappings are refreshed so that the new level's color becomes available without rebuilding the logger.

Parameters

- **name** (*str*) – Name of the custom log level. Dynamic attribute lookup is case-insensitive because requested method names are normalized before registry lookup.
- **color** (*str*) – Color name or formatter-compatible color specification associated with the new log level.
- **severity** (*int*) – Integer severity used when emitting records for the custom level.

Return type

None

Example

Register and use a custom level:

```
logger.add_log_level(  
    name="NOTICE",  
    color="cyan",  
    severity=35,  
)  
  
logger.notice("Service is ready")
```

log(*level, message, *arguments, **keyword_arguments*)

Emit a log record using a level name or numeric severity.

The supplied level is resolved to an integer severity before the record is forwarded to the underlying `logging.Logger`.

Integer levels are accepted directly. String levels are resolved through the application's `LogLevelRegistry`, allowing both standard and custom registered levels to be used.

If `stacklevel` is not supplied in `keyword_arguments`, SpectraLog adds its default stack level so that source information refers to the calling application code.

Parameters

- **level** (*str* / *int*) – Registered log-level name or integer logging severity.
- **message** (*str*) – Log message or formatting template to emit.

- ***arguments** (*object*) – Optional positional values used for deferred logging interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.log()`.

Return type

None

debug(*message*, **arguments*, ***keyword_arguments*)

Emit a DEBUG-level log record.

The message and optional interpolation arguments are forwarded to the underlying `logging.Logger`. SpectraLog automatically supplies its default `stacklevel` unless the caller explicitly provides one.

Parameters

- **message** (*str*) – DEBUG-level log message or formatting template.
- ***arguments** (*object*) – Optional positional values used for deferred message interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.debug()`.

Return type

None

info(*message*, **arguments*, ***keyword_arguments*)

Emit an INFO-level log record.

The message and optional interpolation arguments are forwarded to the underlying `logging.Logger`. SpectraLog automatically supplies its default `stacklevel` unless the caller explicitly provides one.

Parameters

- **message** (*str*) – INFO-level log message or formatting template.
- ***arguments** (*object*) – Optional positional values used for deferred message interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.info()`.

Return type

None

warning(*message*, **arguments*, ***keyword_arguments*)

Emit a WARNING-level log record.

The message and optional interpolation arguments are forwarded to the underlying `logging.Logger`. SpectraLog automatically supplies its default `stacklevel` unless the caller explicitly provides one.

Parameters

- **message** (*str*) – WARNING-level log message or formatting template.
- ***arguments** (*object*) – Optional positional values used for deferred message interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.warning()`.

Return type

None

error(*message*, **arguments*, ***keyword_arguments*)

Emit an ERROR-level log record.

The message and optional interpolation arguments are forwarded to the underlying `logging.Logger`. SpectraLog automatically supplies its default `stacklevel` unless the caller explicitly provides one.

Parameters

- **message** (*str*) – ERROR-level log message or formatting template.
- ***arguments** (*object*) – Optional positional values used for deferred message interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.error()`.

Return type

None

critical(*message*, **arguments*, ***keyword_arguments*)

Emit a CRITICAL-level log record.

The message and optional interpolation arguments are forwarded to the underlying `logging.Logger`. SpectraLog automatically supplies its default `stacklevel` unless the caller explicitly provides one.

Parameters

- **message** (*str*) – CRITICAL-level log message or formatting template.
- ***arguments** (*object*) – Optional positional values used for deferred message interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.critical()`.

Return type

None

exception(*message*, **arguments*, ***keyword_arguments*)

Emit an ERROR-level log record with exception information.

This method delegates to `logging.Logger.exception()` and is intended to be called while handling an active exception. Python’s logging machinery includes the current exception traceback by default.

SpectraLog automatically supplies its default `stacklevel` unless the caller explicitly provides one.

Parameters

- **message** (*str*) – Log message or formatting template describing the exception.
- ***arguments** (*object*) – Optional positional values used for deferred message interpolation.
- ****keyword_arguments** (*Any*) – Additional keyword arguments forwarded to `logging.Logger.exception()`.

Return type

None

Example

Log an exception from an exception handler:

```
try:
    perform_operation()
except RuntimeError:
    logger.exception("Operation failed")
```

shutdown()

Shut down resources owned by the application logger.

Stops the multiprocessing logging runtime when one is configured, allowing queued records to be processed and the runtime's resources to be released.

Shutdown is idempotent. Subsequent calls after the first successful shutdown return immediately without attempting to stop the runtime again.

This method is automatically registered with `atexit` during logger construction, but it may also be called explicitly when deterministic logging cleanup is required.

Return type

None

Logger Construction

```
class spectralog.core.builder.ApplicationLoggerBuilder(configuration, console_handler_factory,
                                                    rich_console_handler_factory,
                                                    file_handler_factory,
                                                    multiprocessing_handler_factory,
                                                    syslog_handler_factory,
                                                    log_file_path_resolver)
```

Build and configure the standard-library logger used by SpectraLog.

`ApplicationLoggerBuilder` is responsible for orchestrating construction of the underlying `logging.Logger` used by `ApplicationLogger`.

The builder applies the active `LoggerConfiguration`, resets any existing handlers on the target logger, configures the effective logging level, and attaches the appropriate console, file, multiprocessing, and syslog handlers.

Handler construction is delegated to injected factories rather than performed directly by the builder. This keeps the class focused on composition and lifecycle orchestration while allowing individual handler implementations to remain independently configurable and testable.

Console output is always configured. When Rich console configuration is present, the Rich handler factory is used; otherwise the standard console handler factory is selected.

When persistent logging is enabled, the log file path is resolved before file output is configured. Depending on the `multiprocessing_safe` setting, records are written either through a direct file handler or through SpectraLog's multiprocessing logging runtime.

When syslog configuration is present, a syslog handler is attached in addition to the configured console and file-related handlers.

Before applying a new configuration, all handlers currently attached to the named logger are removed and closed. Logger propagation is disabled after configuration to prevent duplicate records from being processed by ancestor loggers.

The `build()` method returns a `LoggerBuildResult` containing the configured logger together with metadata required by `ApplicationLogger`, including the resolved log file path, whether the log file is new or empty, and the optional multiprocessing logging runtime.

Parameters

- **configuration** (*LoggerConfiguration*)
- **console_handler_factory** (*ConsoleHandlerFactoryProtocol*)
- **rich_console_handler_factory** (*RichConsoleHandlerFactoryProtocol*)
- **file_handler_factory** (*FileHandlerFactoryProtocol*)
- **multiprocessing_handler_factory** (*MultiprocessingHandlerFactoryProtocol*)
- **syslog_handler_factory** (*SyslogHandlerFactoryProtocol*)
- **log_file_path_resolver** (*LogFilePathResolverProtocol*)

__init__ (*configuration, console_handler_factory, rich_console_handler_factory, file_handler_factory, multiprocessing_handler_factory, syslog_handler_factory, log_file_path_resolver*)

Initialize an application logger builder with its configuration and factories.

The builder coordinates the components required to construct a fully configured `logging.Logger`. Handler creation, file-path resolution, multiprocessing setup, and syslog setup are delegated to injected collaborators so that the builder remains responsible only for orchestration.

Parameters

- **configuration** (*LoggerConfiguration*) – The *LoggerConfiguration* that defines the logger level, output destinations, formatting-related options, file persistence, multiprocessing behavior, and optional Rich or syslog integration.
- **console_handler_factory** (*ConsoleHandlerFactoryProtocol*) – Factory used to create the standard console handler when Rich console output is not configured.
- **rich_console_handler_factory** (*RichConsoleHandlerFactoryProtocol*) – Factory used to create the Rich console handler when `configuration.rich_console_configuration` is provided.
- **file_handler_factory** (*FileHandlerFactoryProtocol*) – Factory used to create the direct file handler when file logging is enabled and multiprocessing-safe logging is disabled.
- **multiprocessing_handler_factory** (*MultiprocessingHandlerFactoryProtocol*) – Factory used to create the multiprocessing logging runtime when file logging is enabled together with `multiprocessing_safe=True`.
- **syslog_handler_factory** (*SyslogHandlerFactoryProtocol*) – Factory used to create the syslog handler when a *SyslogConfiguration* is present.
- **log_file_path_resolver** (*LogFilePathResolverProtocol*) – Resolver used to determine the effective log file path when persistent file logging is enabled.

Return type

None

build(name)

Build and configure the named standard-library logger.

Retrieves the logger identified by `name` from Python’s logging registry, removes and closes any handlers already attached to it, applies the configured logging level, and attaches the handlers required by the active *LoggerConfiguration*.

A Rich console handler is selected when Rich configuration is present; otherwise the standard console handler is used.

When file logging is enabled, the effective log path is resolved and inspected to determine whether it represents a new or empty log file. File output is then configured either through a direct file handler or through SpectraLog's multiprocessing logging runtime, depending on the `multiprocessing_safe` setting.

If syslog configuration is present, a syslog handler is attached in addition to the console and file-related handlers.

Logger propagation is disabled before the completed build result is returned, preventing records from being emitted again by ancestor loggers.

Parameters

name (*str*) – Name of the standard-library logger to retrieve and configure. The name is passed directly to `logging.getLogger()`.

Returns

The completed logger build result containing the configured `logging.Logger`, the resolved log file path when file logging is enabled, whether that file is new or empty, and the optional multiprocessing logging runtime.

Return type

LoggerBuildResult

Note

Building a logger is destructive with respect to handlers already attached to the same named logger. Existing handlers are removed and closed before the new SpectraLog configuration is applied.

class `spectralog.core.factory.ApplicationLoggerBuilderFactory`(*log_level_registry*)

Compose the dependency graph required to build a SpectraLog application logger.

`ApplicationLoggerBuilderFactory` is responsible for constructing and connecting the concrete formatter, handler, path-resolution, and multiprocessing components used by `ApplicationLoggerBuilder`.

The factory centralizes dependency composition so that `ApplicationLoggerBuilder` can focus exclusively on logger construction and orchestration. Components are created with the dependencies they require and then assembled into a fully configured builder for the supplied `LoggerConfiguration`.

A shared `LogLevelRegistry` is retained by the factory and passed to formatter infrastructure so that standard and dynamically registered log levels use a consistent set of severity and color definitions.

A single `RelativePathFilter` instance is shared by the standard console, file, and syslog handler factories. This ensures that relative source path information is populated consistently across supported logging destinations.

File formatting is composed through a `FileFormatterResolver` with ordered formatting strategies. JSON formatting is evaluated before plain-text formatting, allowing JSON logging configuration to select the JSON formatter while plain-text formatting remains the fallback strategy.

The factory also assembles the multiprocessing file logging pipeline by combining the queue handler factory with the standard file handler factory. This allows the resulting builder to choose between direct file logging and queue-based multiprocessing logging according to the active configuration.

Calling `create()` constructs a new dependency graph and returns a fully composed `ApplicationLoggerBuilder`. The logger itself is not created until the returned builder's `ApplicationLoggerBuilder.build()` method is invoked.

Parameters

log_level_registry (`LogLevelRegistry`)

__init__(*log_level_registry*)

Initialize the factory with the shared SpectraLog log-level registry.

The supplied `LogLevelRegistry` is reused when constructing formatter infrastructure so that standard and dynamically registered log levels share the same severity and color definitions.

Parameters

log_level_registry (`LogLevelRegistry`) – Registry containing the log levels known to SpectraLog. The same registry is passed to formatter components created by this factory.

Return type

None

create(*configuration*)

Create a fully composed `ApplicationLoggerBuilder`.

Constructs the complete dependency graph required by `ApplicationLoggerBuilder` for the supplied logger configuration.

The factory creates the format builder, relative-path filter, formatter factories, file formatter strategies, handler factories, multiprocessing logging components, syslog handler factory, and log-file path resolver.

A single `RelativePathFilter` instance is shared by the standard console, file, and syslog handler factories so that source-path enrichment is applied consistently across those logging destinations.

File formatting is resolved through ordered strategies. JSON formatting is evaluated before plain-text formatting so that a configured `JsonLoggerConfiguration` selects the JSON formatter when applicable, while plain-text formatting acts as the fallback.

The returned builder is configured with all collaborators required to build a logger, but no logger or handler is created until `ApplicationLoggerBuilder.build()` is called.

Parameters

configuration (`LoggerConfiguration`) – The `LoggerConfiguration` that will be associated with the created builder and used when the builder later constructs the application logger.

Returns

A fully composed logger builder configured with formatter factories, handler factories, multiprocessing support, syslog support, relative path filtering, and log-file path resolution.

Return type

`ApplicationLoggerBuilder`

Models

class `spectralog.core.models.LoggerBuildResult`(*logger, log_file_path, is_new_log_file, multiprocessing_logging_runtime*)

Describe the result of building a configured SpectraLog logger.

`LoggerBuildResult` groups the objects and metadata produced by `ApplicationLoggerBuilder` after logger construction has completed.

The result allows `ApplicationLogger` to receive the configured standard-library logger together with file-related state and the optional multiprocessing logging runtime without requiring the builder to expose its internal construction details.

Instances are immutable and use slots so that build metadata cannot be changed after creation and arbitrary attributes cannot be added dynamically.

Parameters

- **logger** (*Logger*)
- **log_file_path** (*Path* | *None*)
- **is_new_log_file** (*bool*)
- **multiprocessing_logging_runtime** (*MultiprocessingLoggingRuntime* | *None*)

logger

The fully configured `logging.Logger` created or prepared by the application logger builder.

Type

`logging.Logger`

log_file_path

Resolved path of the active log file when persistent file logging is enabled. `None` when file logging is disabled.

Type

`pathlib.Path` | `None`

is_new_log_file

Indicates whether the resolved log file was newly created or was empty when the logger was built. This value is used by `ApplicationLogger` to determine whether a new-log-file warning should be emitted.

Type

`bool`

multiprocessing_logging_runtime

Multiprocessing logging runtime associated with the logger when multiprocessing-safe file logging is enabled. `None` when logging records are written directly without the multiprocessing queue and listener infrastructure.

Type

`spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime` | `None`

Formatting**class spectralog.formatting.format_builder.LogFormatBuilder**

Build SpectraLog format strings from logger configuration.

`LogFormatBuilder` constructs the plain-text format strings used by SpectraLog's console and file formatters.

When an explicit `console_format` or `file_format` is supplied in the active `LoggerConfiguration`, that value is returned unchanged. Otherwise, the builder composes a format from the configured timestamp, severity, source-location, line-number, and message fields.

Console formats wrap the generated plain-text format with colorlog-compatible `%(log_color)s` and `%(reset)s` fields so that the standard SpectraLog console formatter can apply level-specific colors.

File formats remain plain and do not include console color escape fields.

The generated format components are separated with " | ".

build_console_format(configuration)

Build the format string used by the standard colored console formatter.

If `configuration.console_format` is provided, that explicit format string is returned unchanged.

Otherwise, a plain format is generated from the active configuration and wrapped with the `%(log_color)s` and `%(reset)s` fields expected by SpectraLog's color-aware console formatter.

Parameters

configuration (*LoggerConfiguration*) – Logger configuration controlling the generated console format, including timestamp and source-location fields and any explicit custom console format.

Returns

The explicit console format when one is configured, otherwise the generated plain format wrapped with console color fields.

Return type

`str`

build_file_format(*configuration*)

Build the plain-text format string used for file logging.

If `configuration.file_format` is provided, that explicit format string is returned unchanged.

Otherwise, the format is generated from the active logger configuration using the same plain-format construction rules used by the standard console formatter, but without color-related fields.

Parameters

configuration (*LoggerConfiguration*) – Logger configuration controlling the generated file format, including timestamp and source-location fields and any explicit custom file format.

Returns

The explicit file format when one is configured, otherwise the generated plain-text logging format.

Return type

`str`

```
class spectralog.formatting.formatter_factory.LoggerFormatterFactory(format_builder,  
                                                                    log_level_registry)
```

Create SpectraLog formatters for console and plain-text file output.

`LoggerFormatterFactory` centralizes construction of the formatters used by SpectraLog's standard console and plain-text file handlers.

Format strings are delegated to `LogFormatBuilder`, while log-level color mappings are obtained from the shared `LogLevelRegistry`. This keeps formatter creation separate from both format construction and log-level registration.

Console formatting uses `colorlog.ColoredFormatter` so that log records can be rendered with level-specific colors. File formatting uses the standard `logging.Formatter` and therefore does not include terminal color handling.

Both formatter types use the `date_format` configured in `LoggerConfiguration`.

Parameters

- **format_builder** (`LogFormatBuilder`)
- **log_level_registry** (`LogLevelRegistry`)

```
__init__(format_builder, log_level_registry)
```

Initialize the formatter factory with its formatting dependencies.

Parameters

- **format_builder** (`LogFormatBuilder`) – Builder responsible for producing console and file format strings from the active `LoggerConfiguration`.

- **log_level_registry** (`LogLevelRegistry`) – Registry containing the current Spectra-Log log-level definitions and their associated color mappings. The registry colors are supplied to the colored console formatter.

Return type

None

create_console_formatter(*configuration*)

Create the color-aware formatter used by the standard console handler.

The console format string is obtained from `LogFormatBuilder.build_console_format()` and used to construct a `colorlog.ColoredFormatter`.

The formatter receives the current color mapping from `LogLevelRegistry`, allowing both built-in and registered custom log levels to be rendered with their configured colors.

The configured `date_format` is passed through to the formatter, and color reset behavior is enabled so that terminal styling does not leak into subsequent output.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration used to determine the console format and date format.

Returns

A configured `colorlog.ColoredFormatter` suitable for standard SpectraLog console output.

Return type`logging.Formatter`**create_file_formatter**(*configuration*)

Create the formatter used for plain-text file logging.

The file format string is obtained from `LogFormatBuilder.build_file_format()` and used to construct a standard `logging.Formatter`.

Unlike the console formatter, the file formatter does not apply terminal colors. The configured `date_format` is passed directly to the standard logging formatter.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration used to determine the plain-text file format and date format.

Returns

A configured standard-library formatter suitable for plain-text file logging.

Return type`logging.Formatter`**class** `spectralog.formatting.file_formatter_resolver.FileFormatterResolver`(*formatter_strategies*)

Resolve the appropriate file formatter for a logger configuration.

`FileFormatterResolver` selects a file formatter through an ordered sequence of formatter strategies implementing `FileFormatterStrategyProtocol`.

Strategies are evaluated in the order supplied to the resolver. The first strategy whose `FileFormatterStrategyProtocol.supports()` method returns `True` is selected, and that strategy is then used to create the `logging.Formatter`.

The strategy sequence is copied to an immutable tuple during initialization so that later changes to the original collection do not affect formatter resolution behavior.

This design allows SpectraLog to support multiple file output formats, such as JSON and plain text, without coupling the file handler directly to concrete formatter implementations.

Parameters

formatter_strategies (*Sequence* [*FileFormatterStrategyProtocol*])

__init__ (*formatter_strategies*)

Initialize the resolver with an ordered collection of formatter strategies.

The supplied strategy sequence is copied into a tuple and preserved in the same order. Resolution order is significant because the first strategy that reports support for a configuration is selected.

Parameters

formatter_strategies (*Sequence* [*FileFormatterStrategyProtocol*]) – Ordered collection of file formatter strategies. Each strategy is queried through `supports` and may create the formatter when it is the first strategy compatible with the supplied logger configuration.

Return type

None

resolve (*configuration*)

Resolve and create the file formatter for a logger configuration.

Each configured formatter strategy is evaluated in order. When a strategy reports that it supports the supplied `LoggerConfiguration`, its `create` method is called and the resulting formatter is returned immediately.

No later strategies are evaluated after a compatible strategy has been selected.

Parameters

configuration (*LoggerConfiguration*) – Logger configuration used by each strategy to determine whether it supports the requested file logging mode.

Raises

RuntimeError – If none of the configured formatter strategies supports the supplied logger configuration.

Returns

The formatter created by the first compatible file formatter strategy.

Return type

logging.Formatter

class `spectralog.formatting.json_file_formatter_strategy.JsonFileFormatterStrategy` (*json_formatter_factory*)

Select JSON file formatting when JSON logging is configured.

`JsonFileFormatterStrategy` implements the file formatter strategy used for structured JSON logging.

The strategy reports support only when the active `LoggerConfiguration` contains a `JsonLoggerConfiguration`. When selected, formatter creation is delegated to the injected `JsonLoggerFormatterFactoryProtocol`.

This keeps JSON formatter selection separate from formatter construction and allows `FileFormatterResolver` to choose JSON formatting through the same strategy interface used by other file formats.

Parameters

json_formatter_factory (*JsonLoggerFormatterFactoryProtocol*)

__init__ (*json_formatter_factory*)

Initialize the JSON file formatter strategy.

Parameters

json_formatter_factory (`JsonLoggerFormatterFactoryProtocol`) – Factory responsible for creating the JSON `logging.Formatter` from a `JsonLoggerConfiguration`.

Return type

None

supports(*configuration*)

Return whether the supplied configuration enables JSON file logging.

The strategy is considered applicable when `configuration.json_logger_configuration` is not None.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration to evaluate.

Returns

True when JSON logger configuration is present; otherwise False.

Return type

bool

create(*configuration*)

Create the formatter used for JSON file logging.

The JSON-specific configuration is extracted from the supplied `LoggerConfiguration` and passed to the configured JSON formatter factory.

This method expects the strategy to have already been selected through `supports()`. A missing JSON configuration therefore represents invalid strategy usage and results in an exception.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration containing the required `JsonLoggerConfiguration`.

Raises

ValueError – If `configuration.json_logger_configuration` is None.

Returns

The JSON formatter created by the configured JSON formatter factory.

Return type

`logging.Formatter`

class `spectralog.formatting.plain_text_file_formatter_strategy.PlainTextFileFormatterStrategy`(*formatter_fa*

Select plain-text file formatting when JSON logging is not configured.

`PlainTextFileFormatterStrategy` implements the file formatter strategy used for standard plain-text log files.

The strategy reports support when the active `LoggerConfiguration` does not contain a `JsonLoggerConfiguration`. When selected, formatter creation is delegated to the injected `LoggerFormatterFactoryProtocol`.

This strategy typically acts as the plain-text fallback in `FileFormatterResolver` after more specialized strategies, such as JSON formatting, have been evaluated.

Parameters

formatter_factory (`LoggerFormatterFactoryProtocol`)

__init__(*formatter_factory*)

Initialize the plain-text file formatter strategy.

Parameters

formatter_factory (`LoggerFormatterFactoryProtocol`) – Factory responsible for creating the standard plain-text `logging.Formatter` from a `LoggerConfiguration`.

Return type

None

supports(*configuration*)

Return whether the supplied configuration uses plain-text file logging.

The strategy is applicable when `configuration.json_logger_configuration` is None. A present JSON configuration indicates that JSON file formatting should be selected instead.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration to evaluate.

Returns

True when JSON logging is not configured; otherwise False.

Return type

`bool`

create(*configuration*)

Create the formatter used for plain-text file logging.

Formatter creation is delegated to `LoggerFormatterFactoryProtocol.create_file_formatter()` using the supplied `LoggerConfiguration`.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration used to construct the plain-text file formatter.

Returns

The configured formatter returned by the injected formatter factory.

Return type

`logging.Formatter`

class `spectralog.formatting.json_formatter.JsonLoggerFormatter`(*configuration*)

Format log records as single-line JSON objects.

`JsonLoggerFormatter` converts `logging.LogRecord` instances into structured JSON suitable for JSON Lines log files.

Every formatted record contains the log level and rendered message. Additional metadata such as timestamps, logger names, process information, and thread information is included according to the active `JsonLoggerConfiguration`.

Exception information is included automatically when the log record contains `exc_info`.

The formatter serializes each record with `json.dumps()` using `ensure_ascii=False` so that Unicode characters are preserved. Values that are not directly JSON serializable are converted through `str`.

Each call to `format()` returns one JSON object as a string, making the formatter suitable for line-oriented .jsonl log files.

Parameters

configuration (`JsonLoggerConfiguration`)

__init__(*configuration*)

Initialize the JSON formatter with its metadata configuration.

Parameters

configuration (`JsonLoggerConfiguration`) – Configuration controlling which optional metadata fields are included in each JSON log record.

Return type

None

format(record)

Format a log record as a JSON string.

The supplied `logging.LogRecord` is first converted into a structured dictionary by `_create_log_data()`. The resulting mapping is then serialized with `json.dumps()`.

Unicode characters are preserved because `ensure_ascii` is disabled, and values that are not directly JSON serializable are converted using `str`.

Parameters

record (`LogRecord`) – Log record to serialize.

Returns

A single JSON object representing the supplied log record.

Return type

`str`

class `spectralog.formatting.json_logger_formatter_factory.JsonLoggerFormatterFactory`

Create JSON formatters for SpectraLog structured file logging.

`JsonLoggerFormatterFactory` encapsulates construction of `JsonLoggerFormatter` instances.

The factory accepts a `JsonLoggerConfiguration` and returns a formatter configured with the requested JSON metadata options. This keeps formatter creation separate from file-format strategy selection and allows JSON formatting to be consumed through `JsonLoggerFormatterFactoryProtocol`.

The returned formatter is exposed through the standard `logging.Formatter` abstraction so that handler and strategy components do not need to depend directly on the concrete JSON formatter type.

create(configuration)

Create a JSON formatter from the supplied configuration.

Constructs a `JsonLoggerFormatter` using the provided `JsonLoggerConfiguration`. The configuration determines which optional metadata fields are included in each structured log record.

Parameters

configuration (`JsonLoggerConfiguration`) – JSON logging configuration controlling optional fields such as timestamps, logger names, process information, and thread information.

Returns

A configured `JsonLoggerFormatter` suitable for structured JSON file logging.

Return type

`logging.Formatter`

class `spectralog.formatting.relative_path_filter.RelativePathFilter(project_root=None)`

Enrich log records with a source path relative to a project root.

`RelativePathFilter` adds a `relative_path` attribute to each `logging.LogRecord` processed by the filter. SpectraLog formatters can then reference this value through `%(relative_path)s` when source-path display is enabled.

The project root may be supplied explicitly. When omitted, the current working directory is resolved and used as the root.

If the log record's absolute source path is located beneath the configured project root, `relative_path` contains the path relative to that root. If the source file lies outside the project root, the filter falls back to the record's file name.

The resulting path is normalized to POSIX-style separators so that formatted log output remains consistent across operating systems.

This filter never rejects a log record; its purpose is to enrich the record with additional path metadata before formatting.

Parameters

project_root (*str* | *Path* | *None*)

__init__ (*project_root=None*)

Initialize the relative-path filter.

The supplied project root is converted to a `pathlib.Path` and resolved to an absolute path. When no project root is provided, the resolved current working directory is used.

Parameters

project_root (*str* | *Path* | *None*) – Optional directory used as the base for relative source-path calculation. Strings and `pathlib.Path` instances are accepted. When *None*, `pathlib.Path.cwd()` is used. Defaults to *None*.

Return type

None

filter (*record*)

Add normalized relative source-path information to a log record.

The record's absolute `pathname` is resolved and compared with the configured project root. When the source path is contained within that root, the relative path is stored on the record as `relative_path`.

If the source path cannot be expressed relative to the configured project root, the record's `filename` is used instead.

The stored value uses POSIX-style path separators regardless of the host operating system.

The method always returns `True`, so records are enriched but never filtered out.

Parameters

record (*LogRecord*) – Log record whose source-path metadata should be enriched.

Returns

Always `True`, allowing the log record to continue through the logging pipeline.

Return type

bool

Handlers

class `spectralog.handlers.console_handler_factory.ConsoleHandlerFactory` (*formatter_factory*,
relative_path_filter)

Create the standard SpectraLog console logging handler.

`ConsoleHandlerFactory` constructs and configures the standard `logging.StreamHandler` used for console output when Rich console logging is not enabled.

Formatter creation is delegated to `LoggerFormatterFactory`, while source-path enrichment is provided through a shared `RelativePathFilter`.

The created handler uses the effective logging level from `LoggerConfiguration`, applies the configured color-aware console formatter, and attaches the relative-path filter before being returned.

This factory is responsible only for constructing and configuring the console handler. Attaching the handler to the application logger is handled by `ApplicationLoggerBuilder`.

Parameters

- **formatter_factory** (`LoggerFormatterFactory`)
- **relative_path_filter** (`RelativePathFilter`)

__init__(*formatter_factory, relative_path_filter*)

Initialize the console handler factory with its dependencies.

Parameters

- **formatter_factory** (`LoggerFormatterFactory`) – Formatter factory responsible for creating the console formatter from the active `LoggerConfiguration`.
- **relative_path_filter** (`RelativePathFilter`) – Filter that enriches log records with the `relative_path` attribute used by SpectraLog source-path formatting.

Return type

None

create(*configuration*)

Create and configure the standard console logging handler.

A console formatter is first created from the supplied `LoggerConfiguration`. A `logging.StreamHandler` is then constructed and configured with the effective logger level, the generated formatter, and the shared `RelativePathFilter`.

The returned handler is ready to be attached to a `logging.Logger`.

Parameters

configuration (`LoggerConfiguration`) – Logger configuration controlling the effective logging level and console formatting behavior.

Returns

A configured `logging.StreamHandler` suitable for standard SpectraLog console output.

Return type

`logging.Handler`

class `spectralog.handlers.file_handler_factory.FileHandlerFactory`(*file_formatter_resolver, relative_path_filter*)

Create rotating file handlers for SpectraLog persistent logging.

`FileHandlerFactory` constructs and configures the `logging.handlers.RotatingFileHandler` used for persistent SpectraLog output.

Formatter selection is delegated to the configured `FileFormatterResolverProtocol`, allowing the handler to support multiple file formats such as plain text and JSON without depending directly on a concrete formatter implementation.

The created handler uses UTF-8 encoding, applies the effective logging level from `LoggerConfiguration`, and configures file rotation through the `max_bytes` and `backup_count` settings.

A shared `RelativePathFilter` is attached to each file handler so that formatters requiring `relative_path` source metadata can access it.

This factory is responsible only for constructing and configuring the file handler. Attaching the handler to a logger, or placing it behind a multiprocessing queue, is handled by higher-level SpectraLog components.

Parameters

- **file_formatter_resolver** ([FileFormatterResolverProtocol](#))
- **relative_path_filter** ([RelativePathFilter](#))

__init__(*file_formatter_resolver*, *relative_path_filter*)

Initialize the file handler factory with its formatting dependencies.

Parameters

- **file_formatter_resolver** ([FileFormatterResolverProtocol](#)) – Resolver responsible for selecting and creating the appropriate file formatter for the active [LoggerConfiguration](#).
- **relative_path_filter** ([RelativePathFilter](#)) – Filter that enriches log records with the `relative_path` attribute used by SpectraLog source-path formatting.

Return type

None

create(*configuration*, *log_file_path*)

Create and configure a rotating file logging handler.

The appropriate file formatter is first resolved from the supplied [LoggerConfiguration](#). A [logging.handlers.RotatingFileHandler](#) is then created for `log_file_path` using UTF-8 encoding.

The handler's rotation policy is configured from `max_bytes` and `backup_count`. Its logging threshold is set to the effective configuration level, the resolved formatter is attached, and the shared [RelativePathFilter](#) is added before the handler is returned.

Parameters

- **configuration** ([LoggerConfiguration](#)) – Logger configuration controlling the effective logging level, formatter selection, maximum file size, and retained backup count.
- **log_file_path** ([Path](#)) – Resolved path of the log file to which records should be written.

Returns

A configured [logging.handlers.RotatingFileHandler](#) ready to be attached directly to a logger or used by SpectraLog's multiprocessing logging infrastructure.

Return type

[logging.Handler](#)

class `spectralog.handlers.rich_console_handler_factory`.**RichConsoleHandlerFactory**

Create Rich-based console handlers for SpectraLog.

[RichConsoleHandlerFactory](#) constructs and configures the [rich.logging.RichHandler](#) used when Rich console output is enabled.

The handler's logging threshold is derived from the active [LoggerConfiguration](#), while presentation behavior such as timestamp, level, source path, traceback rendering, and Rich markup is controlled through [RichConsoleConfiguration](#).

A message-only [logging.Formatter](#) is attached to the Rich handler so that Rich remains responsible for rendering its own time, level, path, and traceback columns without duplicating SpectraLog's standard plain-text formatting.

This factory is responsible only for creating and configuring the Rich handler. Attaching the handler to the application logger is handled by [ApplicationLoggerBuilder](#).

create(*logger_configuration*, *rich_configuration*)

Create and configure a Rich console logging handler.

Constructs a `rich.logging.RichHandler` using the effective logging level from `logger_configuration` and the presentation options defined by `rich_configuration`.

A standard `logging.Formatter` containing only `%(message)s` is attached to the handler. This allows Rich to render its own structural elements such as timestamps, levels, source paths, and tracebacks without duplicating fields produced by SpectraLog's plain-text formatter.

Parameters

- **logger_configuration** (*LoggerConfiguration*) – General logger configuration providing the effective logging level and date format.
- **rich_configuration** (*RichConsoleConfiguration*) – Rich-specific configuration controlling timestamp visibility, level visibility, source-path display, enhanced traceback rendering, and Rich markup handling.

Returns

A configured `rich.logging.RichHandler` suitable for SpectraLog console output.

Return type

`logging.Handler`

```
class spectralog.handlers.syslog_handler_factory.SyslogHandlerFactory(formatter_factory,  
                                                                    relative_path_filter)
```

Create syslog handlers for SpectraLog network logging.

`SyslogHandlerFactory` constructs and configures the `logging.handlers.SysLogHandler` used when syslog output is enabled.

The factory combines SpectraLog's general `LoggerConfiguration` with a `SyslogConfiguration`. General logger configuration determines the effective logging level and plain-text formatting behavior, while the syslog configuration defines the destination host, port, facility, and socket type.

Syslog records use SpectraLog's standard file formatter rather than the color-aware console formatter so that terminal-specific color escape sequences are not transmitted to the syslog server.

A shared `RelativePathFilter` is attached to the handler so that format strings containing `%(relative_path)s` can be used safely for syslog output.

This factory is responsible only for constructing and configuring the syslog handler. Attaching the handler to the application logger is handled by `ApplicationLoggerBuilder`.

Parameters

- **formatter_factory** (*LoggerFormatterFactoryProtocol*)
- **relative_path_filter** (*RelativePathFilter*)

```
__init__(formatter_factory, relative_path_filter)
```

Initialize the syslog handler factory with its dependencies.

Parameters

- **formatter_factory** (*LoggerFormatterFactoryProtocol*) – Formatter factory used to create the plain-text formatter applied to outgoing syslog records.
- **relative_path_filter** (*RelativePathFilter*) – Filter that enriches log records with the `relative_path` attribute used by SpectraLog source-path formatting.

Return type

None

create(*logger_configuration*, *syslog_configuration*)

Create and configure a syslog logging handler.

A plain-text formatter is created from `logger_configuration` and attached to a `logging.handlers.SysLogHandler`.

The syslog destination address, facility, and socket type are obtained from `syslog_configuration`. The handler's minimum logging level is set from the effective SpectraLog logger level, and the shared `RelativePathFilter` is attached before the handler is returned.

The configured socket type determines the transport used by `logging.handlers.SysLogHandler`, such as UDP with `socket.SOCK_DGRAM` or TCP with `socket.SOCK_STREAM`.

Parameters

- **logger_configuration** (*LoggerConfiguration*) – General SpectraLog configuration controlling the effective logging level and formatter behavior.
- **syslog_configuration** (*SyslogConfiguration*) – Syslog-specific configuration defining the destination host, port, facility, and socket type.

Returns

A configured `logging.handlers.SysLogHandler` ready to be attached to the application logger.

Return type

`logging.Handler`

class `spectralog.handlers.queue_file_handler_factory.QueueFileHandlerFactory`

Create queue handlers for SpectraLog multiprocessing file logging.

`QueueFileHandlerFactory` constructs the `logging.handlers.QueueHandler` used by SpectraLog's multiprocessing-safe logging pipeline.

The queue handler does not write log records to a file directly. Instead, it places eligible records onto the supplied multiprocessing queue so that a separate `logging.handlers.QueueListener` can forward them to the configured file handler.

This separation allows application logging calls to avoid performing the file write directly while preserving the downstream file handler's formatting, rotation, and filtering behavior.

The factory configures the queue handler with the supplied logging threshold before returning it.

create(*log_record_queue*, *log_level*)

Create and configure a queue handler for log-record transport.

Constructs a `logging.handlers.QueueHandler` backed by the supplied multiprocessing queue and configures it with the requested logging level.

Records accepted by this handler are placed onto `log_record_queue` for later processing by SpectraLog's queue listener and downstream file handler.

Parameters

- **log_record_queue** (*Queue*) – Multiprocessing queue used to transport log records from the application logger to the queue listener.
- **log_level** (*int*) – Minimum integer logging severity accepted by the queue handler.

Returns

A configured `logging.handlers.QueueHandler` ready to be attached to the application logger.

Return type

QueueHandler

class `spectralog.handlers.multiprocessing_handler_factory.MultiprocessingHandlerFactory`(*queue_file_handler_factory*, *file_handler_factory*)

Create the queue-based logging infrastructure used for multiprocessing-safe file logging.

`MultiprocessingHandlerFactory` assembles the components required to route log records through a multiprocessing queue before they are written to the configured file handler.

The factory creates a process-compatible log-record queue, a queue handler that publishes records into that queue, a file handler that performs the actual persistent write, and a `logging.handlers.QueueListener` that consumes queued records and forwards them to the file handler.

The listener is configured with `respect_handler_level=True` so that the effective level configured on the downstream file handler remains authoritative.

The resulting components are wrapped in a `MultiprocessingLoggingRuntime`, which owns their lifecycle and is responsible for starting and stopping the listener and releasing queue-related resources.

This factory creates the multiprocessing logging infrastructure but does not start it. Runtime startup is handled by `ApplicationLogger`.

Parameters

- **queue_file_handler_factory** (`QueueFileHandlerFactory`)
- **file_handler_factory** (`FileHandlerFactoryProtocol`)

__init__(*queue_file_handler_factory*, *file_handler_factory*)

Initialize the multiprocessing handler factory with its dependencies.

Parameters

- **queue_file_handler_factory** (`QueueFileHandlerFactory`) – Factory responsible for creating the queue handler that publishes log records into the multiprocessing queue.
- **file_handler_factory** (`FileHandlerFactoryProtocol`) – Factory responsible for creating the downstream file handler that receives records from the queue listener and writes them to the resolved log file.

Return type

None

create(*configuration*, *log_file_path*)

Create the multiprocessing logging runtime for a log file.

A multiprocessing-compatible queue is created first. The configured queue handler factory then creates a handler that publishes log records into that queue using the effective logger level.

A regular file handler is created for the supplied log file path and attached to a `logging.handlers.QueueListener`. The listener consumes queued log records and forwards them to the file handler while respecting the file handler's configured logging level.

The queue handler, queue listener, and multiprocessing queue are packaged into a `MultiprocessingLoggingRuntime` and returned without being started.

Parameters

- **configuration** (`LoggerConfiguration`) – Logger configuration controlling the effective log level, file formatting, rotation settings, and other persistent logging behavior.
- **log_file_path** (`Path`) – Resolved path of the log file that should receive records consumed by the queue listener.

Returns

A runtime containing the queue handler, queue listener, and multiprocessing queue required for queue-based file logging.

Return type

MultiprocessingLoggingRuntime

File Resolution**class** `spectralog.files.log_file_path_resolver.LogFilePathResolver`

Resolve the effective file-system path used for SpectraLog file logging.

`LogFilePathResolver` determines where SpectraLog should write persistent log output based on the active `LoggerConfiguration`.

The resolver ensures that the configured logs directory exists before constructing the final file path. When a custom `log_file_name` is supplied, that name is used relative to the configured logs directory. When no custom name is supplied, a daily file name is generated from the current local date.

JSON logging changes the effective file extension to `.jsonl`. Plain-text logging uses the supplied custom extension unchanged, or `.log` for automatically generated daily files.

The resolver is responsible only for determining the log file path and creating the top-level logs directory. It does not create or open the log file itself.

resolve(*configuration*)

Resolve the configured log file path and ensure the logs directory exists.

The configured logs directory is obtained from `LoggerConfiguration.resolved_logs_directory` and created recursively when necessary.

If `configuration.log_file_name` is provided, a custom log file path is resolved. Otherwise, a daily log file name based on the current date is generated.

Parameters

configuration (*LoggerConfiguration*) – Logger configuration containing the logs directory, optional custom file name, and optional JSON logging configuration.

Returns

The resolved log file path that should be used for persistent logging.

Return type

Path

Log Levels**class** `spectralog.levels.log_level.LogLevel`(*name, severity, color*)

Represent a registered SpectraLog log-level definition.

`LogLevel` stores the normalized name, numeric severity, and console color associated with a single logging level.

Instances are immutable and use slots, making each level definition stable after creation and preventing arbitrary attributes from being added dynamically.

The registry uses this model for both SpectraLog's built-in levels and custom levels registered at runtime.

Parameters

- **name** (*str*)
- **severity** (*int*)

- **color** (*str*)

name

Normalized log-level name, typically uppercase, such as "INFO" or "NOTICE".

Type

str

severity

Integer logging severity associated with the level.

Type

int

color

Colorlog-compatible color name used by compatible SpectraLog console formatters.

Type

str

Example

Represent a custom level definition:

```
log_level = LogLevel(
    name="NOTICE",
    severity=35,
    color="cyan",
)
```

class spectralog.levels.log_level_registry.LogLevelRegistry

Manage SpectraLog log-level definitions, validation, and color mappings.

LogLevelRegistry stores the standard and custom log levels available to a SpectraLog application logger.

The registry initializes with the standard DEBUG, INFO, WARNING, ERROR, and CRITICAL levels and their default LoggerColor values. Additional levels may be registered dynamically through [register\(\)](#).

Level names are normalized by trimming surrounding whitespace and converting the result to uppercase before validation and lookup. Valid names must begin with a letter and may contain only letters, numbers, and underscores.

Custom colors are validated against the color names supported by colorlog, and severity values must be positive integers. Boolean values are explicitly rejected even though bool is a subclass of int in Python.

Both level names and numeric severities must remain unique within the registry. Registering a duplicate name or reusing a severity already assigned to another level raises SpectraLogLevelAlreadyExistsError.

Registration is protected by an internal lock so that validation against the current registry state and insertion of a new level occur atomically when multiple threads attempt to register levels concurrently.

Registered levels are also published to Python's standard logging subsystem through [logging.addLevelName\(\)](#), allowing their numeric severities to be associated with the same normalized names outside the registry.

The [colors](#) property exposes the current mapping between registered level names and their configured color values for use by compatible console formatters.

__init__()

Initialize the registry with SpectraLog's standard log levels.

Creates the internal synchronization lock and level mapping, then registers the built-in DEBUG, INFO, WARNING, ERROR, and CRITICAL definitions using their standard-library severities and default SpectraLog colors.

Return type

None

property colors: `dict[str, str]`

Return the current log-level color mapping.

Builds a dictionary from the levels currently stored in the registry, mapping each normalized level name to its configured color value.

A new dictionary is returned on every access, so modifying the returned mapping does not directly mutate the registry.

Returns

Mapping of registered log-level names to colorlog-compatible color values.

Return type

`dict[str, str]`

register(*name*, *color*, *severity*)

Register a custom SpectraLog level.

The supplied name is normalized by trimming surrounding whitespace and converting it to uppercase. The normalized name, color, and severity are then validated before registration.

Registration is performed while holding the registry lock. Within that critical section, the method verifies that neither the normalized name nor the numeric severity is already assigned to another registered level.

The level name is also registered with Python's standard logging subsystem through `logging.addLevelName()` before the resulting `LogLevel` is stored in the registry.

Parameters

- **name** (*str*) – Name of the level to register. Surrounding whitespace is removed and the name is converted to uppercase before validation.
- **color** (*str*) – Colorlog-compatible color name used by SpectraLog's colored console formatter.
- **severity** (*int*) – Positive integer logging severity associated with the new level.

Raises

- **`InvalidSpectraLogLevelNameError`** – If the normalized name is empty, does not begin with a letter, or contains characters other than letters, numbers, and underscores.
- **`InvalidSpectraLogColorError`** – If `color` is not present in colorlog's supported escape-code mapping.
- **`InvalidSpectraLogLevelSeverityError`** – If `severity` is not an integer, is a boolean value, or is less than or equal to zero.
- **`SpectraLogLevelAlreadyExistsError`** – If the normalized name is already registered or the supplied severity is already assigned to another registered level.

Returns

The normalized and registered log-level definition.

Return type
LogLevel

get(*name*)

Return a registered log level by name.

The supplied name is normalized by trimming surrounding whitespace and converting it to uppercase before lookup.

Parameters
name (*str*) – Name of the log level to retrieve.

Raises
SpectraLogLevelNotFoundError – If no registered level matches the normalized name.

Returns
 The matching registered log-level definition.

Return type
LogLevel

contains(*name*)

Return whether a log-level name is registered.

The supplied name is normalized using the same rules as registration and lookup before membership is checked.

Parameters
name (*str*) – Name of the log level to check.

Returns
 True when the normalized name exists in the registry; otherwise False.

Return type
bool

Multiprocessing Runtime

```
class spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime(queue_handler,
                                                                                       queue_listener,
                                                                                       log_record_queue)
```

Manage the lifecycle of SpectraLog's multiprocessing logging pipeline.

`MultiprocessingLoggingRuntime` owns the queue handler, queue listener, and multiprocessing queue used for queue-based file logging.

The runtime provides explicit `start()` and `stop()` operations and tracks whether the listener is currently running or whether the runtime has already been permanently closed.

Lifecycle operations are protected by a reentrant lock so that state changes remain consistent when multiple threads attempt to start, stop, or inspect the runtime concurrently.

Starting the runtime launches the configured `logging.handlers.QueueListener`. Stopping the runtime first stops the listener when necessary, then closes the queue handler and multiprocessing queue, waits for the queue's feeder thread to terminate, and marks the runtime as closed.

A closed runtime cannot be started again. Both `start()` and `stop()` are safe to call repeatedly without duplicating lifecycle operations.

Parameters

- **queue_handler** (*QueueHandler*)

- **queue_listener** (*QueueListener*)
- **log_record_queue** (*Queue*)

__init__ (*queue_handler*, *queue_listener*, *log_record_queue*)

Initialize the multiprocessing logging runtime.

Parameters

- **queue_handler** (*QueueHandler*) – Queue handler attached to the application logger. Accepted log records are placed onto the multiprocessing queue for asynchronous processing.
- **queue_listener** (*QueueListener*) – Listener responsible for consuming log records from the queue and forwarding them to the configured downstream handlers.
- **log_record_queue** (*Queue*) – Multiprocessing queue used to transport log records between the queue handler and queue listener.

Return type

None

property queue_handler: *Handler*

Return the queue handler used by the multiprocessing logging runtime.

The application logger attaches this handler instead of attaching the file handler directly when multiprocessing-safe file logging is enabled.

Returns

The queue handler that publishes log records to the multiprocessing logging queue.

Return type

logging.Handler

property is_running: *bool*

Return whether the queue listener is currently running.

The runtime state is read while holding the internal lock to keep lifecycle inspection synchronized with concurrent start and stop operations.

Returns

True when the queue listener has been started and has not yet been stopped; otherwise False.

Return type

bool

start()

Start the multiprocessing queue listener.

The listener is started only when the runtime is not already running and has not previously been closed.

Repeated calls while the runtime is already running have no effect. Calls after *stop()* has permanently closed the runtime also have no effect.

The lifecycle transition is protected by the runtime's internal reentrant lock.

Return type

None

stop()

Stop the multiprocessing logging runtime and release its resources.

If the queue listener is currently running, it is stopped first so that queued records can be processed before queue resources are released.

The queue handler is then closed, followed by the multiprocessing queue. The queue's feeder thread is joined to ensure that its background resources have terminated before shutdown completes.

After cleanup, the runtime is marked as permanently closed and cannot be started again.

This method is idempotent. Calling it after the runtime has already been closed returns immediately without repeating cleanup operations.

Return type

None

Protocols

The following protocols define internal dependency boundaries used throughout SpectraLog.

class `spectralog.core.protocols.LoggerBuilder(*args, **kwargs)`

Define the interface for objects that build configured application loggers.

Implementations are responsible for constructing or preparing a named `logging.Logger` and returning the resulting logger together with any associated file and multiprocessing runtime metadata.

The protocol allows `ApplicationLogger` to depend on logger-building behavior without depending on a concrete builder implementation.

class `spectralog.core.protocols.LoggerFormatterFactoryProtocol(*args, **kwargs)`

Define the interface for creating SpectraLog console and file formatters.

Implementations produce formatters from a `LoggerConfiguration` while encapsulating the concrete formatting strategy used for each output type.

Console and file formatting are exposed separately because they may require different format strings, color handling, or formatter implementations.

class `spectralog.core.protocols.JsonLoggerFormatterFactoryProtocol(*args, **kwargs)`

Define the interface for creating JSON log formatters.

Implementations create a `logging.Formatter` from a `JsonLoggerConfiguration`, allowing JSON formatter construction to be used through an abstraction rather than a concrete formatter factory.

class `spectralog.core.protocols.FileFormatterStrategyProtocol(*args, **kwargs)`

Define the strategy interface for selecting and creating file formatters.

A file formatter strategy determines whether it supports a given `LoggerConfiguration` and, when selected, creates the corresponding `logging.Formatter`.

This protocol allows file formatting behavior to be resolved through ordered strategies such as JSON and plain-text formatting.

class `spectralog.core.protocols.FileFormatterResolverProtocol(*args, **kwargs)`

Define the interface for resolving the appropriate file formatter.

Implementations inspect the supplied `LoggerConfiguration`, select an applicable file formatter strategy, and return the resulting `logging.Formatter`.

The resolver hides strategy-selection details from handler factories that only need the final formatter.

class `spectralog.core.protocols.LogFilePathResolverProtocol(*args, **kwargs)`

Define the interface for resolving the effective log file path.

Implementations derive the final `pathlib.Path` used for persistent logging from a `LoggerConfiguration`.

Path resolution may include directory creation, automatic file naming, custom file names, and extension normalization depending on the active logging configuration.

class `spectralog.core.protocols.ConsoleHandlerFactoryProtocol(*args, **kwargs)`

Define the interface for creating the standard console logging handler.

Implementations create and configure a `logging.Handler` suitable for console output using the supplied `LoggerConfiguration`.

The returned handler is expected to contain any formatter, level, and filter configuration required for standard SpectraLog console logging.

class `spectralog.core.protocols.RichConsoleHandlerFactoryProtocol(*args, **kwargs)`

Define the interface for creating Rich-based console logging handlers.

Implementations create a `logging.Handler` using both the general `LoggerConfiguration` and the Rich-specific `RichConsoleConfiguration`.

This abstraction allows the logger builder to select Rich console output without depending directly on the concrete Rich handler implementation.

class `spectralog.core.protocols.FileHandlerFactoryProtocol(*args, **kwargs)`

Define the interface for creating persistent file logging handlers.

Implementations create and configure a `logging.Handler` for the resolved log file path using the supplied `LoggerConfiguration`.

The concrete handler may provide behavior such as file rotation, formatter selection, filtering, and encoding while remaining hidden behind this protocol.

class `spectralog.core.protocols.MultiprocessingHandlerFactoryProtocol(*args, **kwargs)`

Define the interface for creating multiprocessing-safe logging infrastructure.

Implementations construct a `MultiprocessingLoggingRuntime` for the resolved log file path and active `LoggerConfiguration`.

The resulting runtime is responsible for coordinating queue-based logging components used when SpectraLog's multiprocessing-safe file logging mode is enabled.

class `spectralog.core.protocols.SyslogHandlerFactoryProtocol(*args, **kwargs)`

Define the interface for creating syslog logging handlers.

Implementations create and configure a `logging.Handler` using the general `LoggerConfiguration` together with a `SyslogConfiguration`.

This abstraction isolates syslog transport and handler construction from the application logger builder.

3.12 Architecture

3.12.1 Overview

SpectraLog is designed around a small public API and a set of internal components with clearly separated responsibilities.

The main construction flow is:

```
CreateSpectraLogger
  |
  v
LoggerConfiguration
  |
  v
```

(continues on next page)

(continued from previous page)

```

ApplicationLoggerBuilderFactory
    |
    v
ApplicationLoggerBuilder
    |
    +--> Console Handler
    |
    +--> File Handler
    |
    +--> Multiprocessing Runtime
    |
    +--> Syslog Handler
    |
    v
ApplicationLogger

```

3.12.2 Public API

Applications normally interact with SpectraLog through:

- `CreateSpectraLogger`
- `get_logger`
- `disable_application_logging`

The public API intentionally hides most formatter, handler, and runtime construction details.

3.12.3 Application Logger

`ApplicationLogger` is the process-local logging facade.

It is responsible for:

- exposing standard logging methods;
- supporting dynamically registered custom log levels;
- managing the process-local singleton;
- starting the optional multiprocessing runtime;
- shutting down logging resources;
- forwarding log calls to the underlying standard-library logger.

3.12.4 Logger Configuration

`LoggerConfiguration` contains the complete configuration used to build the logger.

It defines behavior such as:

- effective logging level;
- console formatting;
- file formatting;
- log directory and file name;
- log rotation;

- JSON logging;
- Rich console output;
- syslog output;
- multiprocessing-safe file logging.

3.12.5 Builder Factory

`ApplicationLoggerBuilderFactory` composes the concrete dependencies required to build an application logger.

It creates and connects:

- format builders;
- formatter factories;
- formatter strategies;
- handler factories;
- relative-path filtering;
- file-path resolution;
- multiprocessing logging infrastructure;
- syslog infrastructure.

This keeps object construction centralized rather than spreading dependency creation throughout the package.

3.12.6 Application Logger Builder

`ApplicationLoggerBuilder` orchestrates construction of the underlying `logging.Logger`.

Its responsibilities include:

- retrieving the named logger;
- removing and closing existing handlers;
- setting the configured logging level;
- selecting the standard or Rich console handler;
- configuring file logging;
- configuring multiprocessing logging;
- configuring syslog logging;
- disabling logger propagation;
- returning the completed `LoggerBuildResult`.

3.12.7 Formatting

Formatting responsibilities are separated from handler creation.

LogFormatBuilder

Builds plain-text console and file format strings.

LoggerFormatterFactory

Creates standard colored console and plain-text file formatters.

JsonLoggerFormatter

Serializes log records into structured JSON.

JsonLoggerFormatterFactory

Creates JSON formatter instances.

FileFormatterResolver

Chooses the appropriate file formatter strategy.

JsonFileFormatterStrategy

Selects JSON formatting when JSON logging is configured.

PlainTextFileFormatterStrategy

Selects plain-text formatting when JSON logging is not configured.

3.12.8 File Formatter Strategy

File formatting uses a strategy-based design.

The resolution flow is:

```

LoggerConfiguration
  |
  v
FileFormatterResolver
  |
  +--> JsonFileFormatterStrategy
  |
  +--> PlainTextFileFormatterStrategy
  |
  v
logging.Formatter

```

Strategies are evaluated in order.

JSON formatting is evaluated before plain-text formatting so that JSON configuration takes precedence when enabled.

3.12.9 Handlers

SpectraLog uses dedicated factories for each logging destination.

ConsoleHandlerFactory

Creates the standard color-aware console handler.

RichConsoleHandlerFactory

Creates the Rich-based console handler.

FileHandlerFactory

Creates rotating persistent file handlers.

QueueFileHandlerFactory

Creates queue handlers for multiprocessing file logging.

MultiprocessingHandlerFactory

Assembles the multiprocessing queue, queue handler, file handler, listener, and runtime.

SyslogHandlerFactory

Creates network syslog handlers.

3.12.10 Relative Path Filtering

RelativePathFilter enriches log records with a `relative_path` attribute.

When a log record originates inside the configured project root, the absolute source path is converted to a relative path.

When the source file lies outside the project root, the file name is used as a fallback.

The filter always allows the record to continue through the logging pipeline.

3.12.11 Log File Resolution

LogFilePathResolver determines the final persistent log file path.

It supports:

- custom log file names;
- automatically generated daily file names;
- plain-text `.log` files;
- JSON Lines `.jsonl` files;
- automatic creation of the configured logs directory.

3.12.12 Log Levels

LogLevelRegistry owns SpectraLog's log-level definitions.

The registry contains the standard levels:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

Custom levels can also be registered dynamically.

Each level is represented by a `LogLevel` containing:

- name;
- severity;
- color.

The registry validates custom names, severities, and color values before registration.

3.12.13 Multiprocessing Logging

When `multiprocessing_safe=True` is enabled, file logging uses a queue-based pipeline.

The flow is:

```
ApplicationLogger
  |
  v
QueueHandler
  |
```

(continues on next page)

(continued from previous page)

```

      v
Multiprocessing Queue
      |
      v
    QueueListener
      |
      v
RotatingFileHandler

```

MultiprocessingLoggingRuntime owns the lifecycle of this pipeline.

It starts the queue listener when the application logger is initialized and stops it during logger shutdown.

The runtime also closes the queue handler and multiprocessing queue and joins the queue feeder thread during cleanup.

3.12.14 Singleton Lifecycle

ApplicationLogger is a process-local singleton.

The intended application flow is:

```

from spectralog import CreateSpectraLogger

logger = CreateSpectraLogger()

```

and then, from other modules:

```

from spectralog import get_logger

logger = get_logger()

```

Once the application logger has been explicitly initialized, attempts to reconfigure it are rejected.

Different Python processes have independent singleton instances because each process has its own memory space.

3.12.15 Testing Support

`disable_application_logging` allows application code to continue using the normal SpectraLog API during tests without creating logging side effects.

The decorator disables the logger build infrastructure for the decorated `unittest.TestCase` class and isolates the `ApplicationLogger` singleton for each test method.

This allows tests to execute code that calls `CreateSpectraLogger` and `get_logger` without creating log files, Rich handlers, syslog handlers, or multiprocessing logging infrastructure.

3.12.16 Dependency Boundaries

SpectraLog uses protocols to define internal dependency contracts.

Examples include:

- `LoggerBuilder`
- `LoggerFormatterFactoryProtocol`
- `FileFormatterStrategyProtocol`
- `FileFormatterResolverProtocol`

- `ConsoleHandlerFactoryProtocol`
- `FileHandlerFactoryProtocol`
- `MultiprocessingHandlerFactoryProtocol`
- `SyslogHandlerFactoryProtocol`

This allows higher-level components to depend on behavior rather than concrete implementations.

3.12.17 Design Goals

The architecture is intended to preserve the following properties:

- small public API;
- clear separation of responsibilities;
- testable components;
- explicit dependency composition;
- interchangeable formatter strategies;
- isolated handler construction;
- predictable singleton lifecycle;
- deterministic logging shutdown;
- compatibility with Python's standard logging system.

3.12.18 See Also

See [Internal API](#) for the generated internal API reference.

See [Public API](#) for the public API reference.

3.13 Contributing

Contributions to SpectraLog are welcome. This document describes the expected development workflow, code-quality requirements, testing strategy, and documentation standards for changes to the project.

3.13.1 Development Setup

Clone the repository and install SpectraLog in editable mode:

```
git clone <repository-url>
cd <repository-directory>
python -m pip install -e .
```

Install development and documentation dependencies as required by the project:

```
python -m pip install -e ".[docs]"
```

If the project defines a separate development dependency group, install that group as well.

3.13.2 Project Structure

The source code follows a `src` layout:

```
src/  
├── spectralog/  
│   ├── api/  
│   ├── configuration/  
│   ├── core/  
│   ├── exceptions/  
│   ├── files/  
│   ├── formatting/  
│   ├── handlers/  
│   ├── levels/  
│   └── runtime/
```

Tests are organized separately from the package source:

```
tests/  
├── unit/  
└── integration/
```

Documentation is maintained under:

```
docs/
```

3.13.3 Architecture

Before making structural changes, review [Architecture](#).

SpectraLog separates responsibilities across configuration models, builders, factories, formatter strategies, handler factories, log-level management, and runtime lifecycle components.

Contributions should preserve these boundaries unless a deliberate architectural change is being introduced.

3.13.4 Code Style

New production code should follow the conventions already used throughout the project.

In particular:

- use explicit type annotations;
- prefer descriptive variable names;
- avoid abbreviated variable names;
- keep classes focused on a single responsibility;
- inject collaborators where appropriate rather than constructing unrelated dependencies inside domain components;
- prefer protocol-based dependencies where an abstraction is useful;
- keep public interfaces small and intentional;
- avoid unnecessary comments when the implementation can be expressed clearly through naming and structure;
- preserve existing return-value style and project formatting conventions.

3.13.5 Type Checking

All new and modified code should remain compatible with the project's static type-checking configuration.

Run the configured type checker before submitting changes.

For example:

```
mypy src tests
```

Do not suppress type errors unless there is a specific and documented reason. Prefer accurate typing, explicit casts, or interface improvements over broad `# type: ignore` directives.

3.13.6 Unit Tests

Unit tests should exercise classes and functions in isolation.

The project uses `unittest`.

Test classes should follow the project's naming convention, for example:

```
class UnitTestLogFilePathResolver(unittest.TestCase):  
    ...
```

Each test method should include a concise one-sentence docstring describing the behavior being verified.

Assertions should include meaningful failure messages where supported by the `unittest` assertion API.

For example:

```
self.assertEqual(  
    actual_value,  
    expected_value,  
    "Expected the resolved log level to match the registered severity.",  
)
```

Mocks should be used for external side effects or collaborator boundaries when the purpose of the test is to verify local behavior.

Avoid mocking the behavior that the test is specifically intended to verify.

3.13.7 Integration Tests

Integration tests should exercise SpectraLog through realistic package usage and should avoid mocking internal package components.

Fresh Python subprocesses are preferred when a scenario requires complete isolation of process-local singleton state, `atexit` registration, logging handlers, or multiprocessing runtime state.

Integration tests should verify observable behavior such as:

- console output;
- log file creation;
- plain-text file contents;
- JSON Lines output;
- custom log-level behavior;
- Rich console integration;

- syslog transmission;
- multiprocessing logging shutdown;
- public API imports;
- logging suppression in tests.

3.13.8 Singleton Behavior

`ApplicationLogger` is a process-local singleton.

Tests should not depend on arbitrary execution order or reuse singleton state between logically independent scenarios.

When full process isolation is required, use a subprocess rather than directly modifying internal singleton state in an integration test.

3.13.9 Logging Tests

Tests involving persistent logging should use temporary directories whenever possible.

For example:

```
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory() as temporary_directory:
    logs_directory = Path(temporary_directory) / "logs"
```

This prevents test runs from leaving log files in the repository.

Tests that should execute application code without producing SpectraLog output may use `spectralog.disable_application_logging()`.

3.13.10 Documentation

Public APIs should include complete docstrings suitable for Sphinx-generated documentation.

SpectraLog uses Google-style docstrings rendered through `sphinx.ext.napoleon`.

Public docstrings should document, where applicable:

- the purpose of the object;
- important lifecycle behavior;
- arguments;
- return values;
- raised exceptions;
- relevant side effects;
- examples;
- important limitations or operational notes.

Do not document behavior that is not guaranteed by the implementation.

If public behavior changes, update the corresponding guide and API reference documentation in the same contribution.

3.13.11 Building the Documentation

Build the HTML documentation with:

```
sphinx-build -b html docs docs/_build/html
```

Before submitting documentation changes, run Sphinx with warnings treated as errors:

```
sphinx-build -W --keep-going -b html docs docs/_build/html
```

The documentation build should complete without warnings.

3.13.12 Public API Changes

Changes to the public package API require additional care.

Examples of public API changes include:

- adding or removing package-root exports;
- changing function parameters or defaults;
- changing configuration fields;
- changing exception types;
- changing file naming behavior;
- changing logging output semantics;
- changing singleton lifecycle behavior.

When modifying public behavior:

1. update implementation tests;
2. update integration tests;
3. update public docstrings;
4. update the relevant user guide;
5. update the API reference if necessary;
6. consider backward compatibility.

Package-root imports should remain intentional. Internal implementation classes should not be exported from `spectralog` unless they are intended to become part of the supported public API.

3.13.13 Exceptions

SpectraLog-specific exceptions should inherit from `spectralog.SpectraLogError`.

Use specific exception types where callers may reasonably need to distinguish different failure conditions.

Avoid replacing meaningful package-specific exceptions with generic exceptions unless the error genuinely represents a standard Python contract violation.

3.13.14 Adding Log Levels

Changes to custom log-level behavior should preserve the registry's validation rules and uniqueness guarantees.

Custom level names are normalized before registration. Names and numeric severities must remain unique within a registry.

Changes to supported colors should remain compatible with the formatter backend used by SpectraLog.

3.13.15 Dependency Changes

Avoid introducing new runtime dependencies unless they provide a clear benefit to package users.

A dependency required only for testing, documentation, linting, or release automation should be placed in the appropriate optional development dependency group rather than the package's runtime dependency set.

3.13.16 Submitting Changes

Before submitting a contribution, verify that:

- unit tests pass;
- integration tests pass;
- static type checking passes;
- documentation builds without warnings;
- new public behavior is documented;
- no generated log files or build artifacts are included unintentionally.

A typical validation sequence may look like:

```
python -m unittest discover -s tests
mypy src tests
sphinx-build -W --keep-going -b html docs docs/_build/html
```

Commit changes in focused units that describe one logical modification at a time.

3.13.17 Pull Requests

A pull request should clearly describe:

- what behavior was changed;
- why the change is needed;
- whether the public API is affected;
- what tests were added or updated;
- whether documentation was updated;
- any compatibility considerations or known limitations.

Changes that alter logging semantics, multiprocessing behavior, file handling, or public API contracts should include corresponding integration coverage.

INDICES

- genindex
- modindex
- search

Symbols

__init__() (*spectralog.core.builder.ApplicationLoggerBuilder* build method), 36
__init__() (*spectralog.core.factory.ApplicationLoggerBuilderFactory* build method), 37
__init__() (*spectralog.core.logger.ApplicationLogger* method), 31
__init__() (*spectralog.formatting.file_formatter_resolver.FileFormatterResolver* method), 42
__init__() (*spectralog.formatting.formatter_factory.LoggerFormatterFactory* method), 40
__init__() (*spectralog.formatting.json_file_formatter_strategy.JsonFileFormatterStrategy* method), 42
__init__() (*spectralog.formatting.json_formatter.JsonLoggerFormatter* method), 44
__init__() (*spectralog.formatting.plain_text_file_formatter_strategy.PlainTextFileFormatterStrategy* method), 43
__init__() (*spectralog.formatting.relative_path_filter.RelativePathFilter* method), 46
__init__() (*spectralog.handlers.console_handler_factory.ConsoleHandlerFactory* method), 47
__init__() (*spectralog.handlers.file_handler_factory.FileHandlerFactory* method), 48
__init__() (*spectralog.handlers.multiprocessing_handler_factory.MultiprocessingHandlerFactory* method), 51
__init__() (*spectralog.handlers.syslog_handler_factory.SyslogHandlerFactory* method), 49
__init__() (*spectralog.levels.log_level_registry.LogLevelRegistry* method), 53
__init__() (*spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime* method), 56
A
add_log_level() (*spectralog.core.logger.ApplicationLogger* method), 32
ApplicationLogger (class in *spectralog.core.logger*), 30
ApplicationLoggerBuilder (class in *spectralog.core.builder*), 35
ApplicationLoggerBuilderFactory (class in *spectralog.core.factory*), 37
B
build() (*spectralog.core.builder.ApplicationLoggerBuilder* method), 36
build_console_format() (*spectralog.formatting.format_builder.LogFormatBuilder* method), 39
build_file_format() (*spectralog.formatting.format_builder.LogFormatBuilder* method), 40
C
color() (*spectralog.levels.log_level.LogLevel* attribute), 53
console() (*spectralog.levels.log_level_registry.LogLevelRegistry* property), 54
console_handler_factory (class in *spectralog.handlers.console_handler_factory*), 46
console_handler_factory_protocol (class in *spectralog.core.protocols*), 57
contains() (*spectralog.levels.log_level_registry.LogLevelRegistry* method), 55
create() (*spectralog.core.factory.ApplicationLoggerBuilderFactory* method), 38
create() (*spectralog.formatting.json_file_formatter_strategy.JsonFileFormatterStrategy* method), 43
create() (*spectralog.formatting.json_logger_formatter_factory.JsonLoggerFormatterFactory* method), 45
create() (*spectralog.formatting.plain_text_file_formatter_strategy.PlainTextFileFormatterStrategy* method), 44
create() (*spectralog.handlers.console_handler_factory.ConsoleHandlerFactory* method), 47
create() (*spectralog.handlers.file_handler_factory.FileHandlerFactory* method), 48
create() (*spectralog.handlers.multiprocessing_handler_factory.MultiprocessingHandlerFactory* method), 51
create() (*spectralog.handlers.queue_file_handler_factory.QueueFileHandlerFactory* method), 50
create() (*spectralog.handlers.rich_console_handler_factory.RichConsoleHandlerFactory* method), 48
create() (*spectralog.handlers.syslog_handler_factory.SyslogHandlerFactory* method), 49

`create_console_formatter()` (*spectra-log.formatting.formatter_factory.LoggerFormatterFactory* method), 41
`create_file_formatter()` (*spectra-log.formatting.formatter_factory.LoggerFormatterFactory* method), 41
`CreateSpectraLogger()` (in module *spectralog*), 20
`critical()` (*spectralog.core.logger.ApplicationLogger* method), 34
D
`debug()` (*spectralog.core.logger.ApplicationLogger* method), 33
`disable_application_logging()` (in module *spectralog*), 22
E
`error()` (*spectralog.core.logger.ApplicationLogger* method), 33
`exception()` (*spectralog.core.logger.ApplicationLogger* method), 34
F
`facility` (*spectralog.SyslogConfiguration* attribute), 27
`FileFormatterResolver` (class in *spectra-log.formatting.file_formatter_resolver*), 41
`FileFormatterResolverProtocol` (class in *spectra-log.core.protocols*), 57
`FileFormatterStrategyProtocol` (class in *spectra-log.core.protocols*), 57
`FileHandlerFactory` (class in *spectra-log.handlers.file_handler_factory*), 47
`FileHandlerFactoryProtocol` (class in *spectra-log.core.protocols*), 58
`filter()` (*spectralog.formatting.relative_path_filter.RelativePathFilter* method), 46
`format()` (*spectralog.formatting.json_formatter.JsonLoggerFormatter* method), 45
G
`get()` (*spectralog.levels.log_level_registry.LogLevelRegistry* method), 55
`get_instance()` (*spectra-log.core.logger.ApplicationLogger* class method), 31
`get_logger()` (in module *spectralog*), 22
H
`host` (*spectralog.SyslogConfiguration* attribute), 26
I
`include_logger_name` (*spectra-log.JsonLoggerConfiguration* attribute), 24
`include_process_information` (*spectra-log.JsonLoggerConfiguration* attribute), 24
`include_thread_information` (*spectra-log.JsonLoggerConfiguration* attribute), 24
`include_timestamp` (*spectra-log.JsonLoggerConfiguration* attribute), 24
`info()` (*spectralog.core.logger.ApplicationLogger* method), 33
`InvalidSpectraLogColorError`, 29
`InvalidSpectraLogLevelNameError`, 28
`InvalidSpectraLogLevelSeverityError`, 28
`is_new_log_file` (*spectra-log.core.models.LoggerBuildResult* attribute), 39
`is_running` (*spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime* property), 56
J
`JsonFileFormatterStrategy` (class in *spectra-log.formatting.json_file_formatter_strategy*), 42
`JsonLoggerConfiguration` (class in *spectralog*), 24
`JsonLoggerFormatter` (class in *spectra-log.formatting.json_formatter*), 44
`JsonLoggerFormatterFactory` (class in *spectra-log.formatting.json_logger_formatter_factory*), 45
`JsonLoggerFormatterFactoryProtocol` (class in *spectralog.core.protocols*), 57
L
`LogFileFilter` (*spectralog.core.logger.ApplicationLogger* method), 32
`LogFilePath` (*spectra-log.core.models.LoggerBuildResult* attribute), 39
`LogFilePathResolver` (class in *spectra-log.files.log_file_path_resolver*), 52
`LogFilePathResolverProtocol` (class in *spectra-log.core.protocols*), 57
`LogFormatBuilder` (class in *spectra-log.formatting.format_builder*), 39
`logger` (*spectralog.core.models.LoggerBuildResult* attribute), 39
`LoggerBuilder` (class in *spectralog.core.protocols*), 57
`LoggerBuildResult` (class in *spectralog.core.models*), 38
`LoggerFormatterFactory` (class in *spectra-log.formatting.formatter_factory*), 40
`LoggerFormatterFactoryProtocol` (class in *spectra-log.core.protocols*), 57

LogLevel (class in *spectralog.levels.log_level*), 52

LogLevelRegistry (class in *spectralog.levels.log_level_registry*), 53

M

markup (*spectralog.RichConsoleConfiguration* attribute), 26

multiprocessing_logging_runtime (*spectralog.core.models.LoggerBuildResult* attribute), 39

MultiprocessingHandlerFactory (class in *spectralog.handlers.multiprocessing_handler_factory*), 51

MultiprocessingHandlerFactoryProtocol (class in *spectralog.core.protocols*), 58

MultiprocessingLoggingRuntime (class in *spectralog.runtime.multiprocessing_logging_runtime*), 55

N

name (*spectralog.levels.log_level.LogLevel* attribute), 53

P

PlainTextFileFormatterStrategy (class in *spectralog.formatting.plain_text_file_formatter_strategy*), 43

port (*spectralog.SyslogConfiguration* attribute), 26

Q

queue_handler (*spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime* property), 56

QueueFileHandlerFactory (class in *spectralog.handlers.queue_file_handler_factory*), 50

R

register() (*spectralog.levels.log_level_registry.LogLevelRegistry* method), 54

RelativePathFilter (class in *spectralog.formatting.relative_path_filter*), 45

resolve() (*spectralog.files.log_file_path_resolver.LogFilePathResolver* method), 52

resolve() (*spectralog.formatting.file_formatter_resolver.FileFormatterResolver* method), 42

rich_tracebacks (*spectralog.RichConsoleConfiguration* attribute), 25

RichConsoleConfiguration (class in *spectralog*), 25

RichConsoleHandlerFactory (class in *spectralog.handlers.rich_console_handler_factory*), 48

RichConsoleHandlerFactoryProtocol (class in *spectralog.core.protocols*), 58

S

severity (*spectralog.levels.log_level.LogLevel* attribute), 53

show_level (*spectralog.RichConsoleConfiguration* attribute), 25

show_path (*spectralog.RichConsoleConfiguration* attribute), 25

show_time (*spectralog.RichConsoleConfiguration* attribute), 25

shutdown() (*spectralog.core.logger.ApplicationLogger* method), 35

socket_type (*spectralog.SyslogConfiguration* attribute), 27

SpectraApplicationLoggerAlreadyInitializedError, 28

SpectraApplicationLoggerReconfigurationError, 28

SpectraLogError, 28

SpectraLogLevelAlreadyExistsError, 28

SpectraLogLevelNotFoundError, 28

start() (*spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime* method), 56

stop() (*spectralog.runtime.multiprocessing_logging_runtime.MultiprocessingLoggingRuntime* method), 56

supports() (*spectralog.formatting.json_file_formatter_strategy.JsonFileFormatterStrategy* method), 43

supports() (*spectralog.formatting.plain_text_file_formatter_strategy.PlainTextFileFormatterStrategy* method), 44

SyslogConfiguration (class in *spectralog*), 26

SyslogHandlerFactory (class in *spectralog.handlers.syslog_handler_factory*), 49

SyslogHandlerFactoryProtocol (class in *spectralog.core.protocols*), 58

W

warning() (*spectralog.core.logger.ApplicationLogger* method), 33